



Applications Cloud Native

Préparé par : **M. Yasser JANAHA**

yasser@janah.me

16 août, 2024

Sommaire

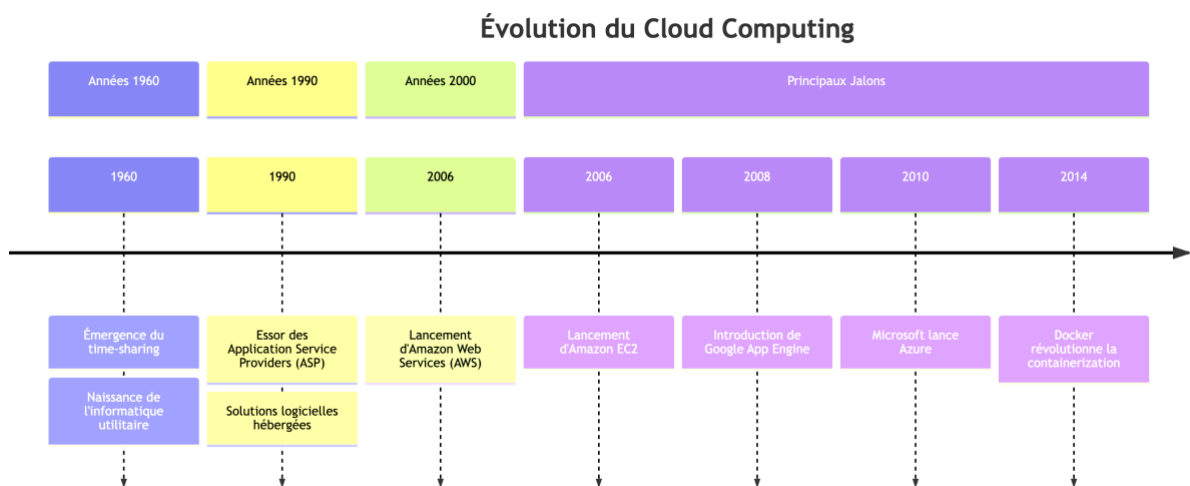
Chapitre 1

Introduction au Cloud Native

I. Concepts de base du Cloud Computing

2. Qu'est-ce que le cloud computing ?

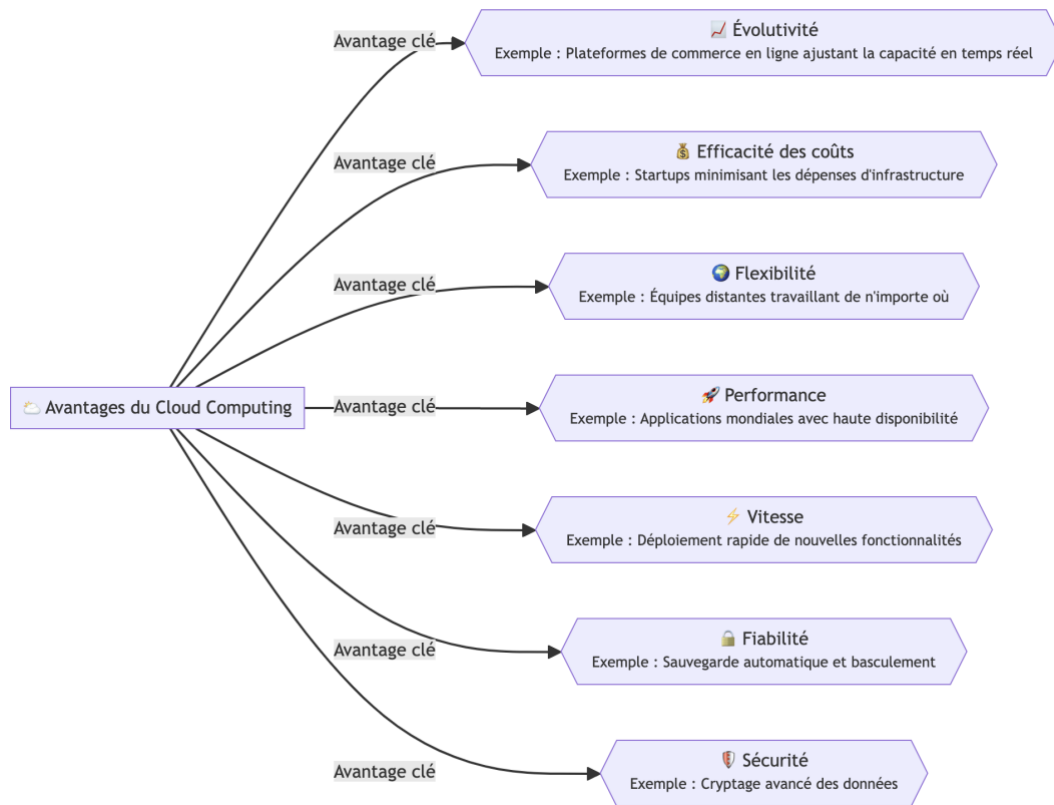
- **Définition :** Le cloud computing est la livraison de divers services, y compris les serveurs, le stockage, les bases de données, les réseaux, les logiciels, les analyses et l'intelligence, sur Internet (« le cloud »). Ces services offrent une innovation plus rapide, des ressources flexibles et des économies d'échelle.
- **Contexte historique:**
 - **Années 1960 :** Le concept de time-sharing (partage de temps) a émergé, permettant à plusieurs utilisateurs de partager des ressources informatiques. Cette période a vu la naissance de l'informatique utilitaire.
 - **Années 1990 :** L'essor des Application Service Providers (ASP) a fourni aux entreprises des solutions logicielles hébergées.
 - **Années 2000 :** Amazon Web Services (AWS) a été lancé en 2006, popularisant le cloud computing en offrant des services d'infrastructure informatique aux entreprises sous forme de services web.
- **Principaux jalons:**
 - **2006:** Lancement d'Amazon Elastic Compute Cloud (EC2).
 - **2008:** Introduction de Google App Engine.
 - **2010:** Microsoft lance Azure.
 - **2014:** Docker révolutionne la containerization.



3. Avantages du cloud computing

- **Évolutivité (Scalability):**
 - Évoluer les ressources vers le haut ou vers le bas en fonction de la demande sans nécessiter d'investissements initiaux significatifs.
 - **Exemple :** Les plateformes de commerce électronique comme Amazon qui évoluent pendant les saisons de pointe.

- **Efficacité des coûts (Cost Efficiency) :**
 - Le modèle de paiement à l'utilisation (pay-as-you-go) permet aux entreprises de ne payer que pour les ressources qu'elles utilisent.
 - **Exemple:** Les startups utilisant des services en cloud pour minimiser les coûts initiaux d'infrastructure.
- **Flexibilité (Flexibility):**
 - Accéder aux ressources et services de n'importe où avec une connexion Internet.
 - **Exemple:** Les équipes distantes collaborant à l'aide d'outils basés sur le cloud comme Google Workspace.
- **Performance (Performance):**
 - Bénéficier du vaste réseau de centres de données sécurisés maintenus par les fournisseurs de cloud.
 - **Exemple:** Haute disponibilité et faible latence pour les applications mondiales.
- **Vitesse (Speed):**
 - Approvisionner rapidement les ressources, souvent en quelques minutes, accélérant le temps de mise sur le marché des applications.
 - **Exemple :** Les développeurs déployant rapidement de nouvelles fonctionnalités à l'aide de plateformes en cloud.
- **Fiabilité (Reliability):**
 - Solutions avancées de sauvegarde des données, de récupération après sinistre et de continuité des activités.
 - **Exemple :** Mécanismes de basculement (failover) automatique dans les bases de données en cloud.
- **Sécurité (Security) :**
 - Fonctionnalités de sécurité avancées et conformité aux réglementations (PCI).
 - **Exemple:** Cryptage des données au repos et en transit, gestion des identités et des accès.



4. Exemples de fournisseurs de cloud

- Amazon Web Services (AWS):



- **Histoire** : Lancé en 2006, AWS est devenu le principal fournisseur de services en cloud, offrant une large gamme de services.
- **Services clés** : EC2 (Elastic Compute Cloud), S3 (Simple Storage Service), RDS (Relational Database Service).
- **Étude de cas** : Netflix utilise AWS pour la diffusion vidéo évolutive.

- Microsoft Azure:



- **Histoire** : Lancé en 2010, Azure est le service de cloud computing de Microsoft, offrant une large gamme de services et d'intégrations.
- **Services clés** : Virtual Machines, Azure SQL Database, Azure Kubernetes Service.
- **Étude de cas** : BMW utilise Azure pour sa plateforme de voiture connectée.

- **Google Cloud Platform (GCP):**



- **Histoire** : Lancé en 2008, GCP offre des services alimentés par la même infrastructure que Google utilise pour ses produits destinés aux utilisateurs finaux.
- **Services clés** : Compute Engine, Cloud Storage, BigQuery.
- **Étude de cas** : Spotify utilise GCP pour le traitement et l'analyse des données.

II. Modèles de services en cloud

- ❑ Le cloud computing offre plusieurs modèles de services qui varient en termes de gestion des ressources, de flexibilité et d'abstraction.

- ❑ **Les modèles principaux sont:** Infrastructure as a Service (**IaaS**), Platform as a Service (**PaaS**), Software as a Service (**SaaS**), Function as a Service (**FaaS**), et Container as a Service (**CaaS**).

1. Infrastructure as a Service (IaaS)

- **Définition** : IaaS fournit des ressources informatiques virtualisées sur Internet. C'est le modèle de service en cloud le plus basique, offrant des infrastructures essentielles telles que des machines virtuelles, des stockages et des réseaux.
- **Exemples** :
 - AWS EC2 : Serveurs virtuels dans le cloud.
 - Google Compute Engine : Machines virtuelles haute performance.
 - Virtual Machines Microsoft Azure : Machines virtuelles avec options de scalabilité.
- **Cas d'utilisation** :
 - Hébergement de sites web et d'applications.
 - Sauvegarde et récupération après sinistre.
 - High-Performance Computing (HPC).

2. Platform as a Service (PaaS)

- **Définition** : PaaS fournit une plateforme permettant aux clients de développer, exécuter et gérer des applications sans gérer l'infrastructure sous-jacente. Il prend en charge le cycle de vie complet des applications.
- **Exemples**:
 - **AWS Elastic Beanstalk** : Déploiement et mise à l'échelle d'applications web.
 - **Google App Engine** : Plateforme gérée pour la création et le déploiement d'applications.
 - **Azure App Services** : Hébergement et exécution d'applications web.
- **Cas d'utilisation**:
 - Développement d'applications web et d'API.
 - Création d'applications mobiles.
 - Simplification des processus DevOps.

3. Software as a Service (SaaS)

- **Définition** : SaaS livre des applications logicielles sur Internet, sur une base d'abonnement. Ces applications sont hébergées et gérées par le fournisseur de services.
- **Exemples**:

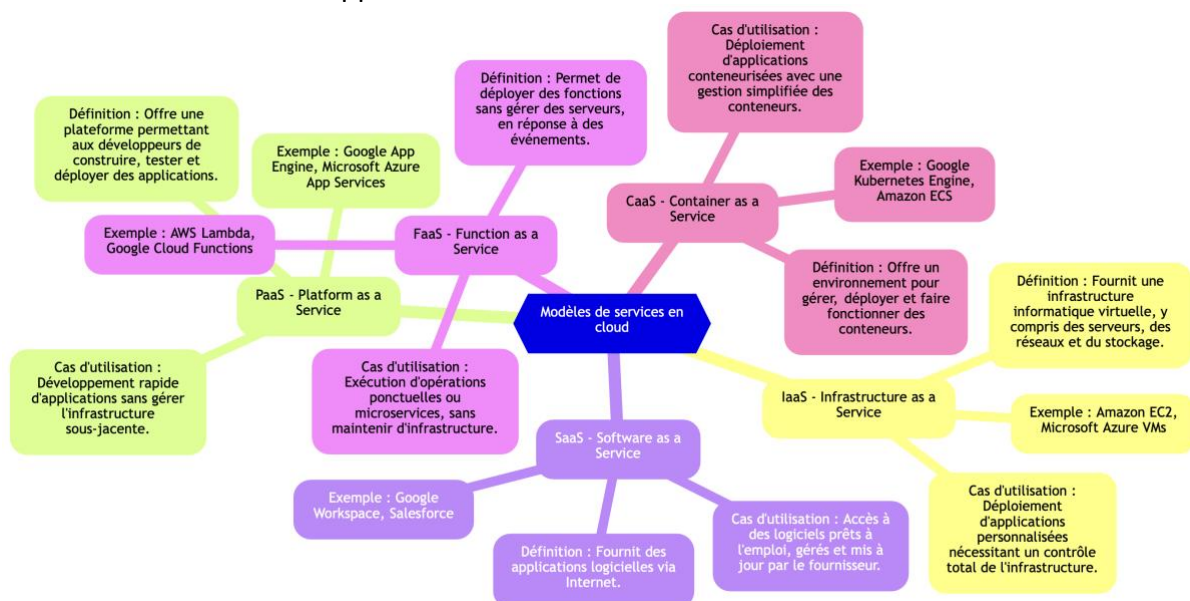
- **Google Workspace** : Outils de productivité en cloud (Gmail, Docs, Drive).
- **Microsoft Office 365** : Versions en ligne des applications Microsoft Office.
- **Salesforce** : Logiciel de gestion de la relation client (CRM).
- **Cas d'utilisation:**
 - Services de messagerie électronique.
 - Gestion de la relation client (CRM).
 - Planification des ressources de l'entreprise (ERP).

4. Function as a Service (FaaS)

- **Définition** : FaaS fournit une plateforme pour développer, exécuter et gérer des fonctionnalités d'application sans maintenir l'infrastructure. Il prend en charge l'exécution événementielle.
- **Exemples:**
 - **AWS Lambda** : Service de calcul sans serveur.
 - **Google Cloud Functions** : Fonctions légères basées sur des événements.
 - **Azure Functions** : Calcul sans serveur basé sur des événements.
- **Cas d'utilisation:**
 - Serverless computing.
 - Traitement des données en temps réel.
 - Architecture de microservices.

5. Container as a Service (CaaS)

- **Définition** : CaaS fournit un environnement géré pour exécuter des applications conteneurisées, facilitant le déploiement, la gestion et la mise à l'échelle des conteneurs.
- **Exemples:**
 - **Google Kubernetes Engine (GKE)**: Service Kubernetes géré.
 - **Azure Kubernetes Service (AKS)**: Orchestration des conteneurs Kubernetes.
 - **AWS Fargate**: Calcul sans serveur pour les conteneurs.
- **Cas d'utilisation:**
 - Orchestration des conteneurs.
 - Déploiement de microservices.
 - Gestion d'applications évolutives.



III. Modèles de déploiement en cloud

- ❑ Le cloud computing offre plusieurs modèles de déploiement qui varient en termes de gestion des ressources, de sécurité, et de flexibilité.
- ❑ **Les modèles principaux sont:** Private Cloud, Public Cloud, Hybrid Cloud, et Multi-Cloud.

1. Private Cloud

- **Définition :** Un nuage privé (private cloud) est une infrastructure de cloud exploitée exclusivement pour une seule organisation. Il peut être géré en interne ou par un tiers et hébergé soit sur place soit à l'extérieur.
- **Avantages :** Sécurité et confidentialité renforcées, plus grand contrôle sur les ressources, conformité aux réglementations.
- **Défis :** Coûts plus élevés, complexité de gestion, évolutivité limitée.
- **Exemples:** VMware vSphere, OpenStack.
- **Cas d'utilisation:**
 - Organisations avec des exigences réglementaires strictes (ex : santé, finance).
 - Entreprises nécessitant une personnalisation et un contrôle élevé sur leur environnement informatique.

2. Public Cloud

- **Définition :** Un nuage public (public cloud) est une infrastructure de cloud disponible pour le grand public ou un large groupe industriel et est détenue par une organisation vendant des services en cloud.
- **Avantages :** Efficacité des coûts, évolutivité, absence de maintenance.
- **Défis :** Préoccupations en matière de sécurité, moins de contrôle sur les ressources.
- **Exemples:** AWS, Azure, Google Cloud.
- **Cas d'utilisation:**
 - Startups et PME cherchant des ressources informatiques rentables.
 - Entreprises nécessitant des ressources évolutives pour des charges de travail variables.

3. Hybrid Cloud

- **Définition :** Un nuage hybride (hybrid cloud) combine des nuages privés et publics, permettant le partage de données et d'applications entre eux. Ce modèle offre une plus grande flexibilité et une optimisation de l'infrastructure existante.
- **Avantages :** Flexibilité, optimisation de l'infrastructure, sécurité et conformité améliorées.
- **Défis :** Complexité de l'intégration, vulnérabilités potentielles en matière de sécurité.
- **Exemples :** IBM Cloud, Microsoft Azure Stack.
- **Cas d'utilisation:**
 - Entreprises avec des systèmes hérités nécessitant une intégration avec des services en cloud modernes.
 - Organisations nécessitant un stockage de données sur site mais des ressources évolutives en cloud public.

4. Multi-Cloud Strategy

- **Définition** : Une stratégie multi-cloud consiste à utiliser plusieurs services en cloud de différents fournisseurs pour éviter la dépendance à un seul fournisseur et optimiser les performances.
- **Avantages** : Flexibilité, atténuation des risques, optimisation des coûts.
- **Défis** : Complexité accrue de gestion, coûts potentiels de transfert de données.
- **Exemples** : Utilisation de AWS pour les ressources de calcul et de Google Cloud pour les analyses de données volumineuses.
- **Cas d'utilisation**:
 - Grandes entreprises cherchant à optimiser les coûts et les performances en utilisant les meilleurs services de plusieurs fournisseurs.
 - Entreprises cherchant à éviter la dépendance à un seul fournisseur de cloud.



5. Comparaison des modèles de déploiement

	Private Cloud	Public Cloud	Hybrid Cloud	Multi-Cloud
Sécurité	Très élevé. Contrôle total sur la sécurité.	Moyenne à élevée. Dépend du fournisseur.	Élevée pour les données sensibles, variable pour le reste.	Variable. Peut être complexe à gérer sur plusieurs clouds.
Coûts	Élevés. Frais initiaux importants et coûts de maintenance.	Bas à moyen. Paiement à l'utilisation.	Coûts initiaux et opérationnels modérés.	Variable. Peut entraîner des coûts supplémentaires pour la gestion.
Évolutivité	Limité par l'infrastructure interne.	Très élevé. Extensible en fonction des besoins.	Flexible, mais peut nécessiter une coordination complexe.	Très élevé, mais nécessite une gestion sophistiquée.
Contrôle	Contrôle total sur l'infrastructure et les données.	Limité. Dépend des politiques du fournisseur.	Contrôle mixte. Forte maîtrise des éléments privés.	Variable. Exige des efforts supplémentaires

				pour un contrôle unifié.
Flexibilité	Moins flexible, dépend de l'infrastructure interne.	Très flexible, rapide à déployer et à ajuster.	Très flexible, combinaison des avantages de private et public clouds.	Très flexible, mais nécessite une gestion coordonnée des différents clouds.

IV. Introduction et Principes des Applications Cloud-Native

1. Qu'est-ce que le Cloud Native ?

- **Définition** : Cloud Native fait référence à une approche de la construction et de l'exécution d'applications qui exploitent pleinement les avantages du modèle de livraison de cloud computing. Les applications Cloud Native sont conçues pour être évolutives, résilientes, gérables et observables.

2. Principes du Cloud Native

- **Définition** : Modèles architecturaux optimisés pour les environnements en cloud. Ils se concentrent sur la création d'applications évolutives, résilientes et gérables.
- **Exemples** :
 - **Microservices** : Décomposition des grandes applications en services plus petits et indépendants.
 - **Service mesh** : Gestion de la communication service à service au sein d'une architecture de microservices.
 - **Event-driven architecture** : Utilisation d'événements pour déclencher et communiquer entre des services découplés.
 - **Containers** : Isolation des applications et de leurs dépendances dans des conteneurs pour une portabilité maximale.
 - **DevOps** : Intégration continue et livraison continue (CI/CD) pour accélérer les cycles de développement.

3. Méthodologie des applications à douze facteurs (Twelve-Factor App Methodology)

- **Introduction** : Un ensemble de meilleures pratiques pour la création d'applications Cloud-Native, se concentrant sur la portabilité et l'évolutivité.
- **Facteurs** :
 - **Codebase** : Une seule base de code suivie dans le contrôle de version, plusieurs déploiements.
 - **Dependencies** : Déclarer explicitement et isoler les dépendances.
 - **Config** : Stocker la configuration dans l'environnement.
 - **Backing Services** : Traiter les services d'arrière-plan comme des ressources attachées.
 - **Build, Release, Run** : Séparer strictement les étapes de construction et d'exécution.
 - **Processes** : Exécuter l'application en tant que processus ou plusieurs processus sans état.
 - **Port Binding** : Exporter les services via le port binding.
 - **Concurrency** : Évoluer via le modèle de processus.
 - **Disposability** : Maximiser la robustesse avec un démarrage rapide et un arrêt gracieux.

- **Dev/Prod Parity** : Garder le développement, la mise en scène et la production aussi similaires que possible.
- **Logs** : Traiter les logs comme des flux d'événements.
- **Admin Processes** : Exécuter les tâches d'administration/de gestion comme des processus ponctuels.

5. Caractéristiques clés des applications Cloud-Native

- **Automatisation** : Pipelines automatisés de build, de test et de déploiement pour assurer une livraison rapide et cohérente des fonctionnalités.
- **Évolutivité** : Conçues pour évoluer horizontalement, gérant des charges variables en ajoutant plus d'instances plutôt que d'évoluer verticalement.
- **Résilience** : Conçues pour résister aux pannes et se rétablir gracieusement, assurant une haute disponibilité.
- **Loose Coupling** : Les services sont faiblement couplés, permettant de les développer, déployer et faire évoluer indépendamment.



Chapitre 2

Développement et sécurisation des API REST avec Node.js et Express.js

I. Introduction à Node.js

- ❑ **Node.js** est une plateforme d'exécution JavaScript côté serveur basée sur le moteur V8 de Google. Il permet d'exécuter du JavaScript en dehors du navigateur, idéal pour créer des applications serveur rapides, évolutives, et capables de gérer des connexions simultanées grâce à son architecture non-bloquante et orientée événements.

1. Histoire et évolution de Node.js

- **Origine de Node.js:**
 - Node.js a été créé par Ryan Dahl en 2009. Avant Node.js, JavaScript était principalement utilisé côté client, dans le navigateur. Dahl a développé Node.js pour permettre aux développeurs d'utiliser JavaScript côté serveur, ce qui a permis de créer des applications plus dynamiques et interactives.
 - **Motivation** : L'idée derrière Node.js était de fournir un moyen plus efficace de gérer les opérations d'entrée/sortie (I/O) dans les serveurs. Ryan Dahl a vu le potentiel de l'Event Loop de JavaScript pour gérer des connexions massives de manière non-bloquante.
- **Évolution de Node.js:**
 - Depuis sa création, Node.js a connu une adoption rapide dans le développement de backend en raison de sa performance et de son efficacité. Les grandes entreprises comme Netflix, LinkedIn, Uber, et Walmart l'ont intégré dans leurs systèmes de production.
 - **Versions notables** :
 - **v0.10.0 (2013)** : Introduction de Streams stables, améliorant la gestion des I/O.
 - **v4.0.0 (2015)** : Node.js se stabilise avec l'unification de Node.js et io.js.
 - **v8.0.0 (2017)** : Améliorations significatives des performances, avec l'introduction de Node.js LTS (Long-Term Support).
- **Contexte historique:**
 - Avant Node.js, les serveurs étaient principalement basés sur des technologies bloquantes, comme PHP avec Apache, où chaque connexion utilisait un thread. Node.js a révolutionné ce modèle en utilisant une architecture orientée événements et non-bloquante, permettant de gérer des milliers de connexions avec un seul thread.

2. Installation et configuration de Node.js

- **Installation de Node.js:**
 - **Via Node Version Manager (NVM):**
 - **Pourquoi NVM ?** : NVM permet de gérer plusieurs versions de Node.js sur une même machine. Ceci est particulièrement utile lorsque différents projets nécessitent des versions différentes.
 - **Installation** :
 - Sur Linux/macOS : Exécution du script d'installation **`curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.0/install.sh | bash`**
 - Sur Windows : Utilisation de nvm-windows, une version adaptée pour Windows.

- **Utilisation :**
 - Pour installer la version la plus récente : ***nvm install node***
 - Pour basculer entre les versions : ***nvm use 14.17.0*** (pour utiliser Node.js version 14.17.0).
- **Installation directe via le site officiel:**
 - Téléchargement du package Node.js à partir de <https://nodejs.org/>.
 - Installation standard via l'installateur pour votre système d'exploitation.
- **Vérification de l'installation:**
 - Exécuter ***node -v*** et ***npm -v*** pour vérifier que Node.js et npm (Node Package Manager) sont bien installés.
- **Configuration de l'environnement:**
 - **Création d'un projet Node.js:**
 - Initialiser un projet avec ***npm init*** pour créer un fichier *package.json* contenant les informations du projet et les dépendances.
 - Le fichier *package.json* permet de gérer les dépendances, les scripts et les métadonnées du projet. C'est un fichier essentiel pour toute application Node.js.
 - Exemple de configuration initiale :

```
{
  "name": "mon-projet-node",
  "version": "1.0.0",
  "description": "Un projet Node.js simple",
  "main": "index.js",
  "scripts": {
    "start": "node index.js"
  },
  "dependencies": {},
  "devDependencies": {},
  "author": "Yasser JANAHA",
  "license": "ISC"
}
```
 - **Installation des dépendances:**
 - Installation des packages nécessaires avec ***npm install package-name***.
 - **Exemple: *npm install express*** pour installer Express.js, un framework populaire pour Node.js.

3. Concepts clés de Node.js

- **Architecture orientée événements (Event-driven architecture):**
 - **Event Loop :** Le cœur de Node.js, qui permet à JavaScript de gérer les opérations asynchrones. L'Event Loop écoute en permanence les événements déclenchés (ex : une requête HTTP) et exécute des callbacks en réponse à ces événements.
 - **Pourquoi c'est important ? :** L'Event Loop permet à Node.js de gérer des milliers de connexions simultanées sans créer un thread pour chaque connexion, contrairement aux serveurs traditionnels.
 - **Exemple :**

- Un serveur qui attend des requêtes HTTP et répond avec un message simple sans bloquer d'autres connexions:

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Bonjour, monde!\n');
});

server.listen(3000, () => {
  console.log('Serveur en écoute sur le port 3000');
});
```

- **I/O non-bloquant (non-blocking I/O):**

- **Principe:** Au lieu d'attendre qu'une opération I/O se termine (comme lire un fichier ou interroger une base de données), Node.js continue d'exécuter d'autres opérations pendant que l'I/O est traité en arrière-plan.
- **Avantages:** Cela améliore la performance en permettant à Node.js de traiter d'autres requêtes pendant qu'une tâche I/O est en cours, réduisant ainsi le temps d'attente pour les utilisateurs.
- **Exemple:** Lire un fichier de manière non-bloquante en utilisant **fs.readFile()**:

```
const fs = require('fs');

fs.readFile('fichier.txt', 'utf8', (err, data) => {
  if (err) {
    console.error(err);
    return;
  }
  console.log(data);
});

console.log('Cette ligne est exécutée en premier, avant la lecture du fichier');
```

- **Applications pratiques:** Utilisé dans des applications en temps réel comme les chats, les serveurs de jeux, et les API qui nécessitent une haute disponibilité et une faible latence.

4. Les fondamentaux de Node.js :

a. Les modules dans Node.js

❑ Introduction aux modules :

- Node.js utilise un système de modules pour organiser le code en morceaux réutilisables. Chaque fichier Node.js est un module.
- **Pourquoi utiliser des modules ? :**
 - Facilite la maintenance et la réutilisation du code.
 - Permet de séparer les préoccupations (Separation of Concerns).

❑ Création d'un module :

- Un module peut être créé en exportant des fonctions ou des objets à partir d'un fichier.
- Exemple de création d'un module math.js :

```
// math.js
function addition(a, b) {
  return a + b;
}

function soustraction(a, b) {
  return a - b;
}

module.exports = {
  addition,
  soustraction
};
```

❑ Exportation et importation des modules :

- **Exportation** : Utilisation de **module.exports** pour rendre les fonctions ou objets disponibles pour d'autres fichiers.
- **Importation** : Utilisation de **require()** pour importer les modules dans un autre fichier.

- Exemple d'importation du module math.js :

```
// app.js
const math = require('./math');

console.log(math.addition(5, 3)); // 8
console.log(math.soustraction(5, 3)); // 2
```

❑ Modules intégrés :

- Node.js fournit des modules intégrés tels que **fs**, **http**, **path**, qui offrent des fonctionnalités puissantes sans avoir besoin d'installer des packages supplémentaires.

b. Gestion de l'asynchronisme

❑ Callbacks asynchrones :

- Les callbacks sont des fonctions passées en argument à d'autres fonctions, qui sont exécutées après l'achèvement d'une opération asynchrone.
- Exemple d'utilisation d'un callback avec **fs.readFile()** :

```
const fs = require('fs');

fs.readFile('fichier.txt', 'utf8', (err, data) => {
  if (err) {
    console.error(err);
    return;
  }
  console.log(data);
});
```

- **Problème du callback hell** : Lorsque les callbacks sont imbriqués de manière excessive, cela rend le code difficile à lire et à maintenir.

❑ Les promesses (Promises) :

- Les promesses sont un moyen de gérer les opérations asynchrones de manière plus lisible et moins imbriquée.
- Une promesse a trois états : **pending**, **fulfilled**, et **rejected**.
- Exemple de création et d'utilisation d'une promesse :

```
const lireFichier = (chemin) => {
  return new Promise((resolve, reject) => {
    fs.readFile(chemin, 'utf8', (err, data) => {
      if (err) {
        reject(err);
      } else {
        resolve(data);
      }
    });
  });
};

lireFichier('fichier.txt')
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

❑ Async/Await :

- **async/await** est une syntaxe moderne qui simplifie l'écriture de code asynchrone. Elle permet d'écrire du code asynchrone de manière synchrone.
- Exemple d'utilisation de **async/await** :

```
const fs = require('fs').promises;

const lireFichier = async (chemin) => {
  try {
    const data = await fs.readFile(chemin, 'utf8');
    console.log(data);
  } catch (err) {
    console.error(err);
  }
};

lireFichier('fichier.txt');
```

c. Gestion des erreurs

❑ Erreurs synchrones vs asynchrones :

- Les erreurs peuvent se produire soit de manière synchrone, soit de manière asynchrone.
- **Gestion des erreurs synchrones** :
 - Utilisation de **try/catch** pour attraper les erreurs dans le code synchrone.
 - Exemple :

```
try {
  const result = faireQuelqueChose();
} catch (error) {
  console.error('Erreur :', error);
}
```

- **Gestion des erreurs asynchrones :**

- **Avec callbacks :** Le premier argument des callbacks est généralement une erreur. Si une erreur se produit, elle est passée à ce premier argument.
- **Avec promesses :** Utilisation de **.catch()** pour gérer les erreurs.
- **Avec async/await :** Les erreurs peuvent être gérées avec try/catch dans un contexte async.
- **Exemple:**

```
const operationAsynchrone = async () => {
  try {
    const result = await someAsyncFunction();
  } catch (error) {
    console.error('Erreur asynchrone :', error);
  }
};
```

d. Gestion des fichiers avec Node.js

- ❑ **Introduction au module *fs* :**

- Le module *fs* de Node.js permet de manipuler le système de fichiers, offrant des méthodes pour lire, écrire, supprimer et renommer des fichiers.
- **Lecture de fichiers :**
 - Utilisation de **fs.readFile()** pour lire un fichier de manière asynchrone.
 - Exemple :

```
const fs = require('fs');

fs.readFile('exemple.txt', 'utf8', (err, data) => {
  if (err) {
    console.error('Erreur de lecture :', err);
    return;
  }
  console.log('Contenu du fichier :', data);
});
```

- **Écriture de fichiers :**

- ❑ Utilisation de **fs.writeFile()** pour écrire dans un fichier de manière asynchrone. Si le fichier n'existe pas, il sera créé ; s'il existe, son contenu sera écrasé.
- ❑ Exemple :

```
const fs = require('fs');

const data = 'Ceci est un exemple de contenu à écrire dans un fichier.';

fs.writeFile('exemple.txt', data, (err) => {
  if (err) {
    console.error('Erreur d\'écriture :', err);
    return;
  }
  console.log('Écriture réussie !');
});
```

- **Manipulation avancée des fichiers :**

- ❑ Ajout de contenu à un fichier :
 - Utilisation de **fs.appendFile()** pour ajouter du contenu à la fin d'un fichier existant.
- ❑ Suppression de fichiers :
 - Utilisation de **fs.unlink()** pour supprimer un fichier.
- ❑ Renommer des fichiers :
 - Utilisation de **fs.rename()** pour renommer ou déplacer un fichier.
- ❑ Exemple de suppression de fichier :

```
fs.unlink('exemple.txt', (err) => {
  if (err) {
    console.error('Erreur de suppression :', err);
    return;
  }
  console.log('Fichier supprimé avec succès !');
});
```

II. Création d'API de base avec Node.js

1. Introduction aux API RESTful

- **Définition des API RESTful:**

- **REST (Representational State Transfer) :** REST est un style d'architecture qui exploite les standards du Web, comme HTTP, pour créer des services Web interopérables. Les API RESTful respectent les principes de REST, ce qui les rend légères, évolutives et faciles à maintenir.
- **Principes clés:**
 - **Statelessness:** Chaque requête du client au serveur doit contenir toutes les informations nécessaires pour comprendre et traiter la requête. Le serveur ne conserve aucune information sur les requêtes précédentes.
 - **Uniform Interface:** Les interactions entre le client et le serveur doivent suivre un protocole standardisé, généralement HTTP, avec des méthodes comme **GET, POST, PUT, DELETE**.
 - **Resource-based:** Les API RESTful utilisent des URI (Uniform Resource Identifiers) pour accéder aux ressources, chaque ressource étant représentée sous forme de JSON ou XML.
 - **Layered System:** L'architecture peut être composée de plusieurs couches, telles que des proxys ou des serveurs de cache, sans que le client en ait connaissance.

- **Pourquoi REST ?:**
 - **Interopérabilité:** REST utilise les standards du Web, ce qui permet à n'importe quel client (navigateur, application mobile, etc.) de consommer une API RESTful.
 - **Simplicité:** Comparé à d'autres protocoles comme SOAP, REST est plus simple à implémenter et à utiliser, notamment grâce à l'utilisation de JSON pour les échanges de données.
 - **Évolutivité:** REST permet de gérer des millions d'utilisateurs simultanés en maintenant une architecture légère et performante.

2. Création d'un serveur HTTP de base avec Node.js

- **Le module http de Node.js :**
 - **Création d'un serveur HTTP :**
 - Le module http de Node.js permet de créer un serveur HTTP qui peut écouter des requêtes sur un port spécifique et répondre à ces requêtes.
 - Exemple de création d'un serveur simple:

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Bonjour, monde!\n');
});

server.listen(3000, () => {
  console.log('Serveur en écoute sur le port 3000');
});
```

- **Explication du code:**
 - **http.createServer():** Crée un serveur qui exécute une fonction de callback chaque fois qu'une requête est reçue.
 - **res.end():** Envoie une réponse au client et termine le processus de requête.
 - **server.listen():** Le serveur commence à écouter sur un port spécifié (ici, le port 3000).
- **Gestion des requêtes et des réponses:**
 - **Requêtes HTTP:**
 - Une requête HTTP comprend une méthode (GET, POST, etc.), un chemin (URI), et des en-têtes.
 - Exemple : Une requête GET sur /:

```
> curl -v "http://127.0.0.1:3000"
* Trying 127.0.0.1:3000...
* Connected to 127.0.0.1 (127.0.0.1) port 3000
> GET / HTTP/1.1
> Host: 127.0.0.1:3000
> User-Agent: curl/8.7.1
> Accept: */*
>
```

- **Réponses HTTP :**

- Une réponse HTTP comprend un code de statut, des en-têtes, et un corps (optionnel).
- Exemple de réponse :

```
* Request completely sent off
< HTTP/1.1 200 OK
< Content-Type: text/plain
< Date: Thu, 22 Aug 2024 14:44:03 GMT
< Connection: keep-alive
< Keep-Alive: timeout=5
< Content-Length: 16
<
Bonjour, monde!
```

- **Flux de requêtes et réponses HTTP :**



3. Développement des endpoints basiques (GET, POST, PUT, DELETE)

- **Méthodes HTTP et leurs utilisations :**

- **GET:** Récupérer des données. Utilisé pour obtenir une représentation d'une ressource.
 - **Exemple:** Récupérer une liste d'utilisateurs.
- **POST:** Créer une nouvelle ressource. Utilisé pour envoyer des données au serveur.
 - **Exemple:** Créer un nouvel utilisateur.
- **PUT:** Mettre à jour une ressource existante. Utilisé pour remplacer une ressource ou créer si elle n'existe pas.
 - **Exemple:** Mettre à jour les informations d'un utilisateur existant.
- **DELETE:** Supprimer une ressource. Utilisé pour supprimer une ressource existante.
 - **Exemple :** Supprimer un utilisateur.



- **Création des routes basiques dans Node.js :**

- **Définir des routes pour les différentes méthodes :**

- **Exemple de code pour créer des routes basiques :**

```
const http = require('http');

const server = http.createServer((req, res) => {
  if (req.method === 'GET' && req.url === '/users') {
    res.statusCode = 200;
    res.setHeader('Content-Type', 'application/json');
    res.end(JSON.stringify({ message: 'Liste des utilisateurs' }));
  } else if (req.method === 'POST' && req.url === '/users') {
    res.statusCode = 201;
    res.setHeader('Content-Type', 'application/json');
    res.end(JSON.stringify({ message: 'Utilisateur créé' }));
  } else if (req.method === 'PUT' && req.url.startsWith('/users/')) {
    res.statusCode = 200;
    res.setHeader('Content-Type', 'application/json');
    res.end(JSON.stringify({ message: 'Utilisateur mis à jour' }));
  } else if (req.method === 'DELETE' && req.url.startsWith('/users/')) {
    res.statusCode = 204;
    res.end();
  } else {
    res.statusCode = 404;
    res.end('Not Found');
  }
});

server.listen(3000, () => {
  console.log('Serveur en écoute sur le port 3000');
});
```

- **Explication du code :**

- Le serveur répond différemment selon la méthode HTTP utilisée et l'URL demandée.
 - Chaque méthode est associée à une action spécifique : récupérer des données, créer, mettre à jour, ou supprimer des ressources.

- **Manipulation des données et réponses JSON :**

- **Utilisation du format JSON :**

- JSON est le format de choix pour les échanges de données via les API RESTful en raison de sa simplicité et de sa compatibilité avec la plupart des langages de programmation.
 - Exemple d'une réponse JSON :

```
{
  "id": 1,
  "nom": "John Doe",
  "email": "johndoe@example.com"
}
```

- **Gérer les données d'entrée :**

- Les données envoyées au serveur via POST ou PUT doivent être analysées (parsed) et validées avant d'être utilisées.
 - Utilisation de **JSON.parse()** pour convertir les données JSON reçues en objets JavaScript manipulables.

III. Introduction à Express.js

1. Histoire des frameworks web

❑ Pourquoi les frameworks sont importants ?

- Un framework fournit une structure standard pour développer et déployer des applications web. Il aide à automatiser des tâches courantes comme la gestion des requêtes HTTP, le routage, la manipulation des vues, et bien plus.
- **Avantages des frameworks :**
 - **Productivité** : Réduction du temps de développement en réutilisant du code standardisé.
 - **Maintenance** : Facilite la gestion du code en fournissant une structure organisée.
 - **Scalabilité** : Les frameworks bien conçus facilitent la montée en charge de l'application.

❑ Introduction à Express.js

- **Historique :**
 - Express.js a été créé par **TJ Holowaychuk** en **2010**. Il est devenu rapidement l'un des frameworks les plus populaires pour Node.js en raison de sa simplicité et de sa flexibilité.
- **Pourquoi Express.js ?**
 - **Minimalisme** : Express est un framework minimaliste qui ne s'impose pas de conventions lourdes, offrant ainsi une grande flexibilité aux développeurs.
 - **Performance** : Construit sur l'architecture légère de Node.js, Express offre des performances élevées pour le développement d'APIs RESTful.
 - **Écosystème** : Large écosystème de middlewares et d'extensions qui permettent de rapidement ajouter des fonctionnalités à une application.

2. Installation et configuration d'Express.js

❑ Installation d'Express.js :

- **Via npm :**
 - Utilisation de la commande suivante pour installer Express.js dans un projet Node.js :
npm install express
 - **Vérification de l'installation :**
 - ❑ Vérifiez que Express est correctement installé en ajoutant express à un fichier de script Node.js.
 - ❑ Exemple de code pour démarrer une application Express de base :


```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello World!');
});

app.listen(3000, () => {
  console.log('Serveur en écoute sur le port 3000');
});
```

❑ Configuration d'une application Express :

▪ Initialisation du projet :

- Créez un nouveau projet Express avec npm init pour générer le fichier package.json.
- Organisez les fichiers du projet avec une structure simple :

```
/mon-projet-express
├─ package.json
├─ app.js
├─ /routes
│   └─ index.js
└─ /views
```

3. Routage avec Express.js

❑ Introduction au routage :

- **Définition** : Le routage est le mécanisme qui permet de définir comment les requêtes HTTP sont traitées par une application web. En Express, chaque route est associée à une méthode HTTP et à un chemin spécifique.

▪ Routes simples :

- Exemple de route GET pour l'accueil d'un site :

```
app.get('/', (req, res) => {
  res.send('Bienvenue sur la page d\'accueil');
});
```

- Route POST pour recevoir des données :

```
app.post('/soumettre', (req, res) => {
  res.send('Données reçues : ' + JSON.stringify(req.body));
});
```

▪ Routes dynamiques :

- **Paramètres de route** : Express permet de créer des routes dynamiques en utilisant des paramètres de route :

```
app.get('/utilisateur/:id', (req, res) => {
  const userId = req.params.id;
  res.send(`Profil de l'utilisateur avec ID: ${userId}`);
});
```

- **Utilisation des query strings** : Les query strings permettent de passer des paramètres optionnels dans l'URL :

```
app.get('/rechercher', (req, res) => {
  const query = req.query.q;
  res.send(`Vous avez recherché : ${query}`);
});
```

4. Middleware avec Express.js

❑ Concept de middleware :

- **Définition** : Un middleware est une fonction qui a accès à l'objet de la requête (req), l'objet de la réponse (res), et à la fonction next dans le cycle de requête-réponse d'une application Express. Le middleware peut exécuter du code, modifier les objets de la requête et de la réponse, terminer le cycle de requête-réponse, et appeler la prochaine fonction middleware dans la pile.
- **Pourquoi utiliser des middlewares ?**
 - Pour gérer des tâches courantes telles que la vérification des autorisations, la gestion des erreurs, la journalisation, et le traitement des données d'entrée.

❑ Exemples de middlewares :

▪ Middleware de journalisation :

- Exemple simple d'un middleware qui journalise chaque requête reçue par le serveur :

```
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next(); // Passe au middleware suivant
});
```

▪ Middleware pour la gestion des erreurs :

- Exemple d'un middleware pour gérer les erreurs globalement dans l'application :

```
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send('Quelque chose s\'est mal passé !');
});
```

▪ Middleware express.json() :

- Pour analyser les requêtes JSON, Express fournit le middleware `express.json()` :
`app.use(express.json());`
- Ce middleware permet à l'application de traiter les données JSON envoyées par le client.

▪ Middlewares tiers (third-party):

- **Body-parser** : Le middleware `body-parser` permet de gérer les données des requêtes POST

```
const bodyParser = require('body-parser');
app.use(bodyParser.urlencoded({ extended: true }));
app.use(bodyParser.json());
```

- **Cookie-parser** : Utilisé pour analyser les cookies dans les requêtes HTTP

```
const cookieParser = require('cookie-parser');
app.use(cookieParser());
```

5. Développement d'une API REST pour la gestion des étudiants à l'OFPPPT Sidi Maarouf

Partie 1 : Utilisation d'un fichier JSON pour les opérations CRUD

a . Contexte et Introduction

Dans cette section, nous allons développer une API REST simple qui permet de gérer les informations des étudiants de l'OFPPPT Sidi Maarouf en utilisant un fichier JSON comme base de données. Ce choix est motivé par sa simplicité et sa pertinence pour les petits projets ou les prototypes où une base de données complète n'est pas nécessaire. Un fichier JSON permet de stocker les données sous un format structuré et lisible, tout en étant facilement manipulable via Node.js.

b. Configuration du projet Node.js avec Express.js

Étapes préliminaires :

- Initialiser un projet Node.js et installer les modules nécessaires (**express** et **fs**).
- Créer une structure de projet adaptée pour gérer les données des étudiants.

c. Implémentation des opérations CRUD avec un fichier JSON

Dans cette partie, nous allons créer une API REST qui permet de **Créer, Lire, Mettre à jour, et Supprimer** (CRUD) les informations des étudiants stockées dans un fichier JSON.

Création d'un étudiant (Create)

- **Endpoint**: POST /students
- **Objectif**: Ajouter un nouvel étudiant dans le fichier JSON students.json.
- **Exemple de code**:

```
app.post('/students', (req, res) => {
  let newStudent = req.body;
  let data = JSON.parse(fs.readFileSync('data/students.json'));
  data.push(newStudent);
  fs.writeFileSync('data/students.json', JSON.stringify(data));
  res.status(201).send('Étudiant ajouté avec succès');
});
```

Lecture des étudiants (Read)

- **Endpoint**: GET /students
- **Objectif**: Récupérer tous les étudiants enregistrés dans le fichier JSON.
- **Exemple de code**:

```
app.get('/students', (req, res) => {
  let data = JSON.parse(fs.readFileSync('data/students.json'));
  res.status(200).json(data);
});
```

Mise à jour d'un étudiant (Update)

- **Endpoint:** GET /students/:id
- **Objectif:** Mettre à jour les informations d'un étudiant existant identifié par son ID.
- **Exemple de code:**

```
app.put('/students/:id', (req, res) => {
  let id = req.params.id;
  let updatedStudent = req.body;
  let data = JSON.parse(fs.readFileSync('data/students.json'));
  let studentIndex = data.findIndex(student => student.id == id);
  if(studentIndex !== -1) {
    data[studentIndex] = updatedStudent;
    fs.writeFileSync('data/students.json', JSON.stringify(data));
    res.status(200).send('Étudiant mis à jour avec succès');
  } else {
    res.status(404).send('Étudiant non trouvé');
  }
});
```

Suppression d'un étudiant (Delete)

- **Endpoint:** DELETE /students/:id
- **Objectif:** Supprimer un étudiant de la liste à partir de son ID.
- **Exemple de code:**

```
app.delete('/students/:id', (req, res) => {
  let id = req.params.id;
  let data = JSON.parse(fs.readFileSync('data/students.json'));
  let newData = data.filter(student => student.id !== id);
  fs.writeFileSync('data/students.json', JSON.stringify(newData));
  res.status(200).send('Étudiant supprimé avec succès');
});
```

d. Test de l'API avec Postman

Utilisation de Postman pour tester les différents endpoints de l'API en envoyant des requêtes HTTP (GET, POST, PUT, DELETE) et en vérifiant les réponses JSON

- Installation via le site officiel (<https://www.postman.com/>)

Partie 2 : Utilisation de MongoDB pour les opérations CRUD

a . Introduction à MongoDB et Mongoose

MongoDB est une base de données NoSQL conçue pour gérer des quantités importantes de données non structurées ou semi-structurées. Mongoose est un **ORM** (Object-Relational Mapping) qui simplifie les interactions avec MongoDB en fournissant un modèle de données

solide et cohérent. Dans cette partie, nous allons étendre l'exemple précédent en utilisant MongoDB pour gérer les informations des étudiants, ce qui est plus adapté pour des applications à plus grande échelle ou nécessitant des fonctionnalités de base de données plus avancées.

b . Configuration de MongoDB avec Mongoose

- Installation de Mongoose: *npm install mongoose*
- Connection à la base de données:

```
const mongoose = require('mongoose');
mongoose.connect('mongodb://localhost/ofppt_students', {
  useNewUrlParser: true,
  useUnifiedTopology: true
})
.then(() => console.log('Connexion à MongoDB réussie'))
.catch(err => console.error('Erreur de connexion à MongoDB', err));
```

c . Création d'un schéma de données pour les étudiants

Définition du schéma des étudiants

- **Objectif:** Structurer les données des étudiants pour les stocker dans MongoDB.
- Exemple de code:

```
const studentSchema = new mongoose.Schema({
  name: String,
  age: Number,
  program: String,
  enrolled: Boolean
});
const Student = mongoose.model('Student', studentSchema);
```

d . Implémentation des opérations CRUD avec MongoDB

Création d'un étudiant (Create)

- Utilisation de *save()*:

```
const newStudent = new Student({ name: 'Ahmed', age: 20, program: 'Développement Web', enrolled: true });
newStudent.save()
.then(student => res.status(201).json(student))
.catch(err => res.status(400).json('Erreur lors de l\'ajout de l\'étudiant'));
```

Lecture des étudiants (Read)

- Utilisation de *find()*:

```
Student.find()
.then(students => res.status(200).json(students))
.catch(err => res.status(500).json('Erreur lors de la récupération des étudiants'));
```

Mise à jour d'un étudiant (Update)

- Utilisation de *findByIdAndUpdate()*:

```
Student.findByIdAndUpdate(req.params.id, { age: 21 }, { new: true })
.then(student => res.status(200).json(student))
.catch(err => res.status(400).json('Erreur lors de la mise à jour de l\'étudiant'));
```

Suppression d'un étudiant (Delete)

- Utilisation de `findByIdAndDelete()`:

```
Student.findByIdAndDelete(req.params.id)
  .then(() => res.status(200).send('Étudiant supprimé avec succès'))
  .catch(err => res.status(500).json('Erreur lors de la suppression de l\'étudiant'));
```

IV. Développement avancé d'API

1. Méthodes d'authentification pour les API

❑ Stateless vs Stateful :

▪ Stateless (JWT) :

- **Définition** : Les authentifications sans état (stateless) ne nécessitent pas que le serveur conserve des informations sur l'utilisateur entre les requêtes. Au lieu de cela, le client envoie un token signé (comme JWT) avec chaque requête, et le serveur valide ce token.
- **Avantages** :
 - **Scalabilité accrue** : Le serveur n'a pas besoin de gérer des sessions, ce qui est idéal pour les architectures distribuées.
 - **Sécurité** : Les tokens JWT sont signés numériquement, ce qui rend difficile leur falsification.
- **Inconvénients** :
 - **Difficulté de révocation** : Une fois émis, un token JWT ne peut pas être facilement révoqué avant sa date d'expiration.

▪ Stateful (Sessions) :

- **Définition** : L'authentification basée sur les sessions nécessite que le serveur conserve les informations de session pour chaque utilisateur connecté. Une session est généralement identifiée par un cookie envoyé par le client.
- **Avantages** :
 - **Facilité de gestion** : Les sessions peuvent être révoquées à tout moment.
 - **Contrôle d'accès granulaire** : Le serveur peut suivre précisément l'état de l'utilisateur.
- **Inconvénients** :
 - **Scalabilité limitée** : Le suivi des sessions sur de nombreux serveurs peut devenir complexe.

❑ Comparaison entre JWT et Sessions :

- **JWT** est idéal pour les applications modernes qui nécessitent une scalabilité horizontale et des architectures distribuées.
- **Sessions** sont plus simples à mettre en place dans des environnements centralisés mais peuvent devenir un goulot d'étranglement dans les architectures à grande échelle.

2. Explication de JWT (JSON Web Tokens)

• Qu'est-ce que JWT ?

JWT, ou JSON Web Token, est un standard ouvert utilisé pour créer des jetons d'accès qui permettent d'échanger des informations de manière sécurisée entre deux parties, généralement un client et un serveur. Le JWT est composé de trois

parties principales (Header, Payload, et Signature), chacune encodée en Base64 et séparée par des points (.)

- **Header** : Contient le type de token et l'algorithme utilisé pour signer.


```
{
  "alg": "HS256",
  "typ": "JWT"
}
```
- **Payload** : Contient les données (claims) encodées en base64, comme l'identité de l'utilisateur et les permissions.


```
{
  "sub": "ID1234",
  "name": "Yasser JANAHA",
  "admin": true
}
```
- **Signature** : Résultat de la signature du Header et du Payload avec une clé secrète.
- **Exemple d'un JWT :**
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJJRDEyMzQ1IiwiaWF0IjoiYXNjaWVzZ2VyeEpBTkFlliwiYWRTaW4iOnRydWV9.3Pc1x0AJPEmDfzK9kfwL2MwsmAD0bW7yZNMxrqXjkos
- **Utilisation de JWT pour l'authentification :**
 - **Émission** : Le serveur émet un JWT après une authentification réussie, que le client stocke (par exemple, dans un cookie ou localStorage).
 - **Validation** : À chaque requête suivante, le client envoie le JWT dans l'en-tête HTTP Authorization. Le serveur valide la signature du JWT pour autoriser ou refuser l'accès.
 - **Exemple en Express.js :**
 - **Émission d'un JWT :**

```
const jwt = require('jsonwebtoken');

app.post('/login', (req, res) => {
  const user = { id: 3 }; // Idéalement, vérifiez les informations d'identification de l'utilisateur ici
  const token = jwt.sign({ user }, 'votre_secret', { expiresIn: '1h' });
  res.json({ token });
});
```


- **Vérification d'un JWT :**

```
const jwt = require('jsonwebtoken');

const verifierToken = (req, res, next) => {
  const token = req.headers['authorization'];
  if (!token) return res.status(403).send('Token requis');

  jwt.verify(token, 'votre_secret', (err, decoded) => {
    if (err) return res.status(403).send('Token invalide');
    req.user = decoded.user;
    next();
  });
};

app.get('/route_protegee', verifierToken, (req, res) => {
  res.send('Bienvenue, utilisateur authentifié !');
});
```

- **Le client doit ajouter le jeton aux en-têtes de la requête :**

Authorization: Bearer

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJJRDEyMzQjLCJuYW1lIjoiaWVWFzc2VyeEpBTkFlliwiYWRTaW4iOnRydWV9.3Pc1x0AJPEmDfzK9kfwL2MwsmAD0bW7yZNMxrqXjkos

3. Sécurisation des API

- **OAuth2 et OpenID Connect :**

- **OAuth2 :**

- **Définition :** OAuth2 est un cadre d'autorisation qui permet aux applications tierces d'obtenir un accès limité à un service HTTP, soit pour le compte d'un utilisateur, soit en tant qu'application elle-même.
- **Flux d'autorisation :** OAuth2 propose plusieurs types de flux (authorization code, client credentials, etc.) adaptés à différents scénarios d'authentification.

- **OpenID Connect :**

- **Définition :** Extension de OAuth2 qui ajoute une couche d'authentification. OpenID Connect permet de vérifier l'identité de l'utilisateur et d'obtenir des informations de profil de base.
- **Exemple d'utilisation :** Connexion via un compte Google ou Facebook sur un site tiers.

- **Intégration dans les applications d'entreprise :**

Dans les applications d'entreprise, plutôt que de mettre en œuvre directement des JWT (JSON Web Tokens), il est courant d'utiliser des solutions centralisées telles que **Keycloak** ou des outils similaires. Ces solutions gèrent l'authentification et l'autorisation de manière sécurisée, en

implémentant les protocoles OAuth2 et OpenID Connect. Elles offrent également des fonctionnalités avancées telles que la gestion des utilisateurs, des rôles, des permissions, et la possibilité de centraliser la gestion des identités. Cela permet de simplifier la sécurisation des API et de réduire les risques liés à la gestion manuelle des jetons, tout en s'intégrant facilement avec d'autres services et applications d'entreprise.

- **Protection des endpoints :**

- **Middleware d'authentification :** En Express.js, les middleware peuvent être utilisés pour protéger des routes spécifiques ou toutes les routes d'une application.
- **Exemple de protection d'une route avec un middleware JWT :**

```
app.get('/data_protegee', verifierToken, (req, res) => {  
  res.json({ message: 'Voici des données protégées' });  
});
```

- **Gestion des rôles et permissions :**
 - Les JWT peuvent inclure des informations sur les rôles et permissions de l'utilisateur dans le payload, ce qui permet de gérer l'accès à certaines fonctionnalités selon le rôle de l'utilisateur.

4. Pratiques avancées pour le développement d'API

- **Rate Limiting :**

- Limiter le nombre de requêtes qu'un client peut envoyer à un serveur sur une période donnée pour éviter les abus et les attaques DDoS.
- **Exemple avec Express Rate Limit :**

```
const rateLimit = require('express-rate-limit');  
  
const limiteur = rateLimit({  
  windowMs: 15 * 60 * 1000, // 15 minutes  
  max: 100 // Limite chaque IP à 100 requêtes par fenêtre de 15 minutes  
});  
  
app.use('/api/', limiteur);
```

- **Logging et Monitoring :**

- Suivre l'activité des API pour déceler les problèmes de performance et détecter les attaques potentielles.
- **Exemples d'outils :** Winston pour le logging, Prometheus pour le monitoring.
- **Exemple de configuration de Winston :**

```
const winston = require('winston');

const logger = winston.createLogger({
  level: 'info',
  format: winston.format.json(),
  transports: [
    new winston.transports.File({ filename: 'erreurs.log', level: 'error' }),
    new winston.transports.File({ filename: 'combined.log' })
  ]
});
```

- **CORS (Cross-Origin Resource Sharing) :**
 - **Problématique :** Les navigateurs appliquent une politique de sécurité appelée Same-Origin Policy, qui empêche les requêtes provenant d'un domaine différent de celui du serveur.
 - **Solution :** Utiliser CORS pour autoriser des domaines spécifiques à accéder aux ressources d'une API.
 - **Exemple d'activation de CORS en Express.js :**

```
const cors = require('cors');
app.use(cors({ origin: 'https://example.com' }));
```

V. Historique du développement des API

1. Évolution des méthodes d'authentification des API

- **Authentification basique (Basic Auth) :**
 - **Principe :** Basic Auth envoie l'identifiant et le mot de passe de l'utilisateur en texte clair dans l'en-tête Authorization de chaque requête HTTP.
 - **Avantages :**
 - Simplicité de mise en œuvre.
 - Largement supporté par les serveurs et les clients HTTP.
 - **Inconvénients :**
 - **Sécurité faible :** Nécessite que les communications soient chiffrées (HTTPS), sinon les informations d'identification peuvent être facilement interceptées.
 - **Non évolutif :** Les informations d'identification sont envoyées à chaque requête, ce qui n'est pas optimal pour des systèmes à grande échelle.
- **Sessions et cookies :**
 - **Principe :** Lorsqu'un utilisateur se connecte, le serveur crée une session et stocke l'ID de session sur le serveur. Un cookie contenant cet ID est envoyé au client pour les requêtes suivantes.
 - **Avantages :**
 - **Contrôle centralisé :** Le serveur peut invalider la session à tout moment.
 - Capacité à gérer des autorisations complexes en maintenant l'état de l'utilisateur.

- **Inconvénients :**
 - **Scalabilité limitée :** Maintenir l'état de session pour chaque utilisateur nécessite de la mémoire et peut poser des problèmes dans les architectures distribuées.
- **Introduction de JWT :**
 - **Besoin de statelessness :** Avec la montée des architectures distribuées et des microservices, il est devenu nécessaire de décentraliser l'authentification. JWT a été introduit pour permettre une authentification stateless, où le client détient son propre état sous forme de token signé.
 - **Comparaison avec les méthodes précédentes :**
 - Contrairement à Basic Auth, JWT encapsule non seulement l'authentification, mais aussi des informations contextuelles (claims).
 - Contrairement aux sessions, JWT ne nécessite pas de stockage côté serveur, ce qui améliore la scalabilité.
- **OAuth2 et OpenID Connect :**
 - **Évolution vers une gestion granulaire des permissions :** OAuth2 et OpenID Connect ont été introduits pour permettre aux utilisateurs de déléguer des autorisations sans partager leurs informations d'identification, et pour permettre aux applications tierces d'accéder aux ressources au nom de l'utilisateur.
 - **Exemple :**
 - Une application tierce qui souhaite accéder aux contacts d'un utilisateur Google sans demander son mot de passe Google utilise OAuth2 pour obtenir un token d'accès limité.

2. Du monolithique aux microservices

- **Architecture monolithique :**
 - **Définition :** Une architecture où toutes les fonctionnalités de l'application sont intégrées dans un seul bloc de code ou une seule application, déployée en tant qu'unité unique.
 - **Avantages :**
 - **Simplicité initiale :** Facilité de développement et de déploiement en phase de démarrage.
 - **Tests unitaires simplifiés :** Le code est centralisé, ce qui facilite les tests unitaires.
 - **Inconvénients :**
 - **Difficulté à évoluer :** Les changements dans une partie du code nécessitent de redéployer toute l'application.
 - **Risques accrus :** Une erreur dans une partie de l'application peut affecter tout le système.
- **Architecture SOA (Service-Oriented Architecture) :**
 - **Transition vers SOA :**
 - Avant les microservices, SOA a introduit le concept de services indépendants communiquant via un bus de services d'entreprise (ESB). Bien que plus modulaire que les monolithes, SOA restait complexe en raison de la gestion de l'ESB et des dépendances interservices.

- **Exemple de SOA** : Une banque avec différents services pour les comptes, les prêts, et les cartes de crédit, tous exposant des interfaces de service via un ESB.
- **Émergence des microservices** :
 - **Définition** : Les microservices sont une évolution de SOA, où chaque service est une application autonome, responsable d'une fonction métier spécifique, et communique avec d'autres services via des API légères.
 - **Avantages** :
 - **Scalabilité indépendante** : Chaque microservice peut être déployé et mis à l'échelle indépendamment des autres.
 - **Résilience** : Un échec dans un microservice n'affecte pas nécessairement les autres services.
 - **Flexibilité technologique** : Chaque microservice peut être développé en utilisant la technologie la plus adaptée à son domaine spécifique.
 - **Inconvénients** :
 - **Complexité accrue** : La gestion de la communication interservices, du déploiement et du monitoring nécessite des outils et une architecture plus sophistiqués.
 - **Exemple d'architecture microservices** : Une application de commerce en ligne avec des microservices séparés pour le panier, le paiement, la gestion des produits, et la gestion des utilisateurs.

3. Impact des changements technologiques sur la sécurité et la scalabilité des API

- **Sécurité dans les architectures monolithiques** :
 - **Modèle centralisé** : La sécurité était souvent gérée par une seule couche d'authentification et d'autorisation dans l'application.
 - **Problèmes** :
 - Une brèche dans la couche de sécurité centrale peut exposer l'ensemble de l'application.
 - La gestion des autorisations était souvent rigide, avec des difficultés à mettre en œuvre des politiques d'accès granulaire.
- **Sécurité dans les microservices** :
 - **Décentralisation de la sécurité** : Avec les microservices, chaque service doit être sécurisé de manière indépendante, ce qui nécessite des mécanismes comme OAuth2, JWT, et les gateways API pour centraliser certaines fonctions de sécurité.
 - **Zero Trust Architecture** : Les microservices adoptent souvent un modèle de sécurité Zero Trust, où chaque requête est authentifiée et autorisée de manière indépendante.
 - **Challenges** :
 - **Gestion des tokens** : Les tokens JWT doivent être bien gérés pour éviter les fuites d'informations ou les usurpations d'identité.
 - **Communication sécurisée entre les services** : Utilisation de TLS, des VPNs, et des proxies sécurisés pour protéger les communications entre microservices.

- **Scalabilité dans les architectures monolithiques :**
 - **Limitations :** Les monolithes sont difficiles à faire évoluer horizontalement. Toute augmentation de la capacité nécessite d'ajouter des ressources à l'ensemble du système, ce qui peut être coûteux et inefficace.
- **Scalabilité dans les microservices :**
 - **Scalabilité indépendante :** Les microservices permettent d'augmenter la capacité de chaque composant individuellement, en fonction des besoins. Par exemple, le microservice de paiement peut être mis à l'échelle indépendamment du microservice de gestion des utilisateurs.
 - **Orchestration :** Utilisation de Kubernetes et autres outils d'orchestration pour automatiser le déploiement et la gestion de la scalabilité des microservices.

Chapitre 3

Architecture et Implémentation des Microservices

I. Introduction aux Microservices

1.1 Historique et évolution : Du Monolithe aux Microservices

1.1.1 Qu'est-ce qu'une architecture monolithique ?

Une architecture monolithique est un modèle de conception dans lequel l'ensemble des composants fonctionnels d'une application sont regroupés au sein d'une seule et même base de code. Cela inclut généralement :

- L'interface utilisateur (UI)
- La logique métier (règles, traitements, contrôleurs)
- L'accès aux données (requêtes vers la base de données)
- Les services auxiliaires (authentification, notifications, gestion des erreurs, etc.)

Tous ces éléments sont compilés, testés et déployés ensemble sous forme d'une unique unité exécutable (projet: Express, Laravel, django, fichier: .jar, .war, ou conteneur Docker unique). Le tout fonctionne dans le même environnement d'exécution (processus, mémoire, base de données partagée).

Exemple concret : Système universitaire monolithique:

Considérons une application utilisée par une université pour gérer toutes ses opérations internes. Cette application intègre les modules suivants :

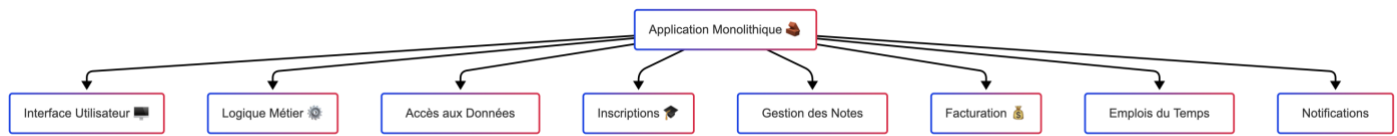
- **Inscription** des étudiants
- **Paie**ment des frais de scolarité
- **Gestion des cours** et des plannings
- **Suivi des notes** et des bulletins
- **Notifications** (emails, SMS, alertes)

Tous ces modules sont développés dans un seul projet, versionnés ensemble et partagent les mêmes ressources (base de données, sessions, logs). Une modification dans le module des paiements oblige à recompiler et redéployer l'ensemble de l'application.

Problèmes associés :

- **Scalabilité limitée** : si un seul module connaît un pic de charge (ex : les inscriptions à la rentrée), il faut répliquer tout le système, même si les autres parties sont peu sollicitées.
- **Manque de résilience** : une panne dans une partie du code (ex : erreur d'accès à la base dans le module des notes) peut entraîner une indisponibilité totale de l'application.
- **Faible flexibilité technologique** : tous les modules doivent utiliser la même technologie, ce qui limite l'adoption de solutions modernes plus adaptées à certaines tâches.
- **Cycle de déploiement complexe** : chaque mise en production nécessite de tester et valider toute l'application, augmentant les délais et le risque d'introduire des bugs ailleurs.
- **Couplage fort** : les modules dépendent fortement les uns des autres, avec un risque de propagation d'erreurs et des interdépendances difficiles à isoler.
- **Difficulté de collaboration** : sur de gros projets, plusieurs équipes doivent travailler sur le même codebase, ce qui complique la gestion des branches, des merges et des tests.

Ce modèle reste pertinent pour des projets simples, ou en phase initiale, mais il devient rapidement un frein dans les systèmes à forte croissance, à haute disponibilité ou soumis à des cycles de mise à jour fréquents.



1.1.2 Évolution vers les architectures distribuées

Avec la montée en complexité des systèmes informatiques, la nécessité de mieux organiser le code, de faciliter les mises à l'échelle, et d'améliorer la résilience a mené à une transition progressive vers des architectures distribuées. Cette évolution a été jalonnée de plusieurs étapes technologiques majeures, chacune apportant ses propres avantages et limitations.

RPC (Remote Procedure Call)

- Le RPC permet à un programme d'exécuter une procédure sur une machine distante comme s'il s'agissait d'un appel local.
- Ce mécanisme est simple et rapide, mais il présente une forte dépendance au réseau.
- Il ne gère pas bien les pannes, les timeouts ou les variations de latence.
- Il repose souvent sur des protocoles propriétaires ou faiblement standardisés, ce qui limite l'interopérabilité.

Limites :

- Faible tolérance aux erreurs réseau
- Couplage fort entre client et serveur
- Difficile à faire évoluer dans des systèmes complexes

CORBA (Common Object Request Broker Architecture)

- CORBA est un standard d'interopérabilité entre objets répartis dans des langages différents et sur des plateformes hétérogènes.
- Il utilise un middleware appelé ORB (Object Request Broker) pour faire communiquer les objets.
- CORBA fournit un IDL (Interface Definition Language) pour décrire les interfaces exposées.

Avantages :

- Forte abstraction
- Support multi-langages

Limites :

- Mise en œuvre complexe
- Courbe d'apprentissage élevée
- Difficulté à déboguer

SOA (Service-Oriented Architecture)

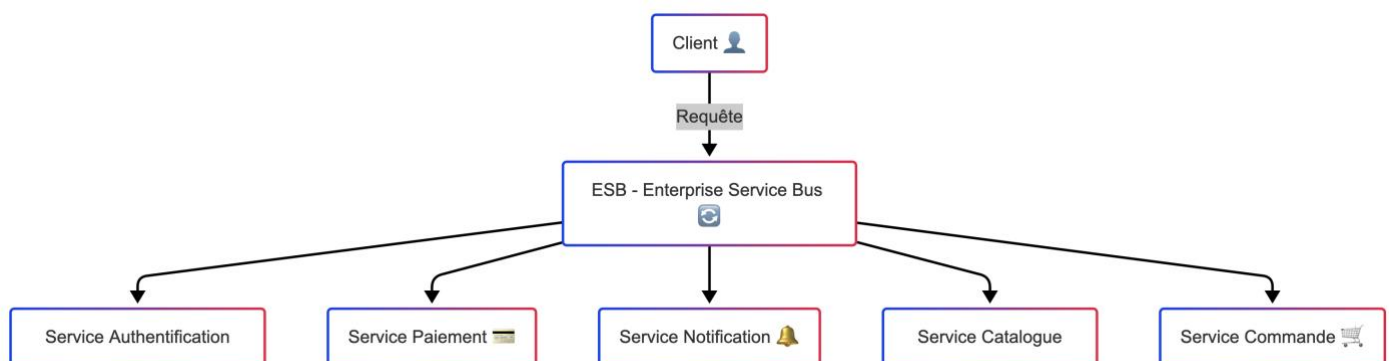
- SOA a introduit la notion de services réutilisables accessibles via un **ESB (Enterprise Service Bus)**, qui centralise la communication.
- Chaque composant ou service représente une brique fonctionnelle du système (ex. : facturation, commande, livraison).

Avantages :

- Réutilisabilité accrue des services métier
- Encapsulation des fonctions
- Séparation des responsabilités

Limites :

- **Dépendance au middleware (ESB)** : l'ESB devient un point de concentration critique (single point of failure).
- **Complexité opérationnelle** : configuration, surveillance, gestion de la charge du bus.
- **Évolution lente** : chaque changement implique une coordination forte entre les services et l'ESB.
- **Couplage indirect** entre services via le bus, freinant l'agilité.



1.1.3 L'émergence des microservices

Les microservices représentent une évolution naturelle de l'architecture SOA (Service-Oriented Architecture), mais adaptée aux exigences modernes de l'informatique distribuée, à savoir la rapidité, l'élasticité, l'automatisation et la résilience. Cette approche est née de la convergence de plusieurs tendances technologiques : la conteneurisation (Docker), l'orchestration (Kubernetes), l'automatisation des livraisons (CI/CD) et la culture DevOps.

Définition

Un **microservice** est une unité logicielle **autonome**, **faiblement couplée** et **fortement cohésive**, qui réalise une fonction métier bien définie. Il expose ses fonctionnalités via une API légère (souvent REST ou gRPC), et gère ses propres données. Contrairement à une architecture monolithique, chaque microservice peut être développé, déployé, mis à l'échelle et supervisé indépendamment des autres.

Caractéristiques fondamentales

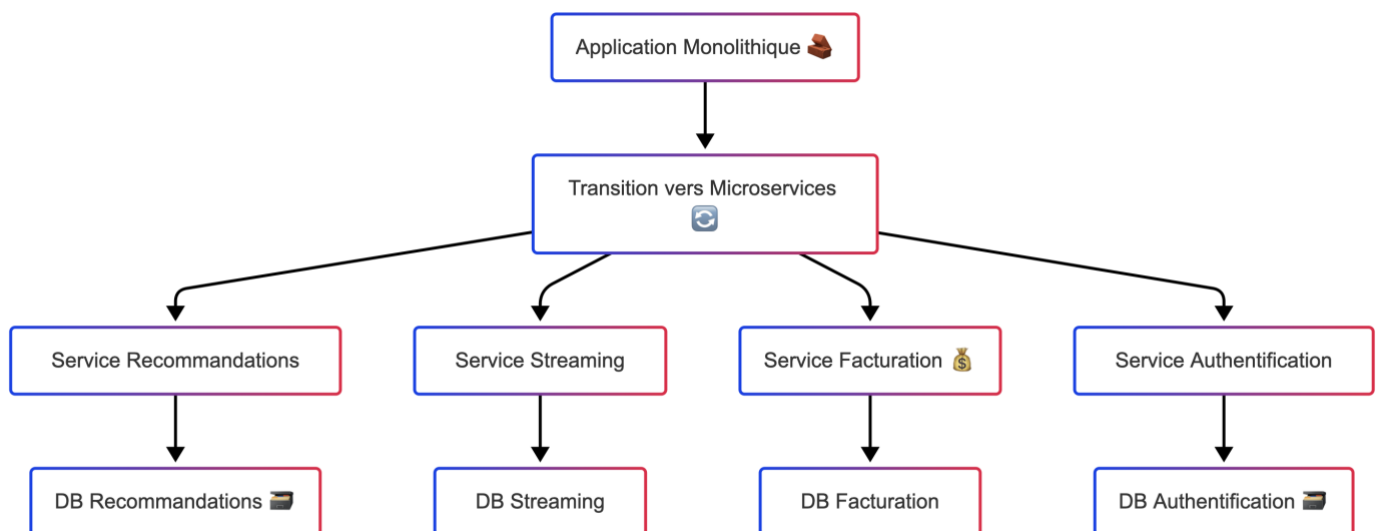
- **Autonomie complète** : chaque microservice a son propre cycle de vie de développement (build, test, déploiement, mise à l'échelle). Cela permet à des équipes indépendantes de travailler en parallèle.
- **Base de données dédiée** : chaque service gère ses propres données, évitant ainsi les dépendances indirectes via une base partagée.
- **Communication légère** : les services échangent des données via des protocoles simples comme HTTP/REST, gRPC ou par des messages asynchrones (Kafka, RabbitMQ).
- **Déploiement indépendant** : chaque service peut être mis en production indépendamment grâce aux pipelines CI/CD, facilitant le delivery continu.
- **Flexibilité technologique** : chaque microservice peut être écrit dans un langage ou un framework adapté à sa tâche spécifique (polyglottisme).
- **Observabilité renforcée** : chaque service doit émettre ses propres métriques, logs et traces pour garantir une surveillance efficace.

Exemple réel : Netflix

Netflix est un pionnier de l'architecture microservices à grande échelle. L'entreprise a migré d'un système monolithique vers une plateforme basée sur des centaines de microservices indépendants, chacun responsable d'un domaine précis.

- **Service de recommandations** : génère des suggestions personnalisées.
- **Service de streaming** : gère la distribution des vidéos.
- **Service de facturation** : traite les paiements et abonnements.
- **Service d'authentification** : gère la sécurité et les sessions utilisateurs.
- **Service de monitoring** : supervise les autres services et la performance globale.

Netflix a aussi développé plusieurs outils open-source comme **Hystrix** (circuit breaker), **Eureka** (Service Discovery) et **Zuul** (API Gateway) pour soutenir cette architecture distribuée.



1.2 Principes fondamentaux des microservices

Les microservices ne sont pas simplement une approche de découpage technique, mais un véritable paradigme architectural fondé sur des principes précis. Ces principes permettent de concevoir des systèmes plus agiles, plus résilients et plus faciles à faire évoluer. Voici les fondements les plus importants :

Principe de responsabilité unique (SRP - Single Responsibility Principle)

Chaque microservice est responsable d'un **seul domaine fonctionnel** ou d'un **sous-domaine métier bien défini**. Cette spécialisation permet d'assurer une forte cohésion interne, une plus grande lisibilité du code, et une plus grande facilité de maintenance et de test.

Ce principe est directement inspiré des bonnes pratiques de conception orientée objet, mais il s'applique ici à l'échelle des services.

Exemple :

- Le service **Commandes** gère uniquement la création, la modification et l'annulation de commandes.
- Le service **Paiement** gère exclusivement les opérations financières, comme la facturation ou la validation de paiements.

En cas d'évolution du métier (par exemple, ajout de paiements en plusieurs fois), seul le service **Paiement** sera concerné.

Scalabilité indépendante

L'architecture microservices permet de mettre à l'échelle uniquement les parties du système soumises à une forte charge, sans impacter les autres. C'est ce que l'on appelle la **scalabilité horizontale ciblée**.

Chaque service peut être répliqué autant de fois que nécessaire pour faire face à une hausse de trafic ou de calculs, en fonction de ses besoins propres.

Exemple :

Lors d'un événement commercial (soldes, black friday), le service **Panier** est particulièrement sollicité. Grâce à l'indépendance des services, il peut être répliqué sur plusieurs instances sans devoir redémarrer les services **Utilisateurs** ou **Produits**.

Résilience

L'un des objectifs majeurs des microservices est d'augmenter la tolérance aux pannes du système global. Si un service échoue, le reste de l'application continue à fonctionner normalement.

Cela est rendu possible grâce à :

- L'isolation des services dans des processus indépendants
- La communication asynchrone

- L'utilisation de mécanismes comme le **circuit breaker**, les **timeouts** ou les **retries**

Exemple :

Si le service **Notifications** tombe en panne, le service **Commandes** continue de prendre des commandes normalement. Une file d'attente peut stocker les événements en attente pour une reprise ultérieure.

Cette capacité à tolérer des erreurs locales sans provoquer de panne globale est cruciale dans les environnements cloud, où l'instabilité réseau ou applicative est fréquente.

1.3 Comparaison Microservices vs Monolithique

Critère	Monolithique	Microservices
Déploiement	Global	Par service
Évolutivité	Limitée	Granulaire et ciblée
Flexibilité	Faible	Élevée (choix techno par service)
Base de données	Unique, partagée	Isolée pour chaque service
Résilience	Panne d'un module = panne globale	Isolement des pannes
Déploiement continu	Difficile	Naturel avec CI/CD

II. Composants des Microservices

2.1 Service Discovery

Objectif

Dans un environnement distribué et dynamique comme ceux du cloud (Kubernetes, ECS, etc.), les services ne sont pas déployés avec des adresses IP statiques. Ils sont souvent créés, arrêtés ou redémarrés automatiquement. Cela signifie que leurs points d'accès (IP, ports) peuvent changer fréquemment. Pour assurer une communication fluide et automatisée entre microservices, on utilise un mécanisme appelé **Service Discovery** (découverte de services).

Il permet :

- d'automatiser la détection des services actifs dans l'écosystème,
- de connecter les microservices sans configuration manuelle,
- d'adapter dynamiquement les appels réseau à l'état réel du système.

Mécanisme

Le fonctionnement repose généralement sur un **registre de services** centralisé, avec deux opérations principales :

1. **Enregistrement (Registration)** : chaque service, au démarrage, annonce sa présence au registre. Il y publie des informations comme : nom du service, adresse IP, port, état (UP/DOWN), et éventuellement des métadonnées.
2. **Résolution (Lookup/Discovery)** : lorsqu'un autre service a besoin de communiquer, il interroge le registre pour obtenir l'adresse actuelle du service cible.

Deux grandes approches de découverte

- **Client-Side Discovery** :
 - Le client est responsable d'interroger le registre, de résoudre l'adresse du service cible, et de faire la requête.
 - Exemples : Netflix Eureka (Spring Cloud Netflix), Consul + Ribbon.
 - Le client contient la logique de répartition de charge (load balancing).

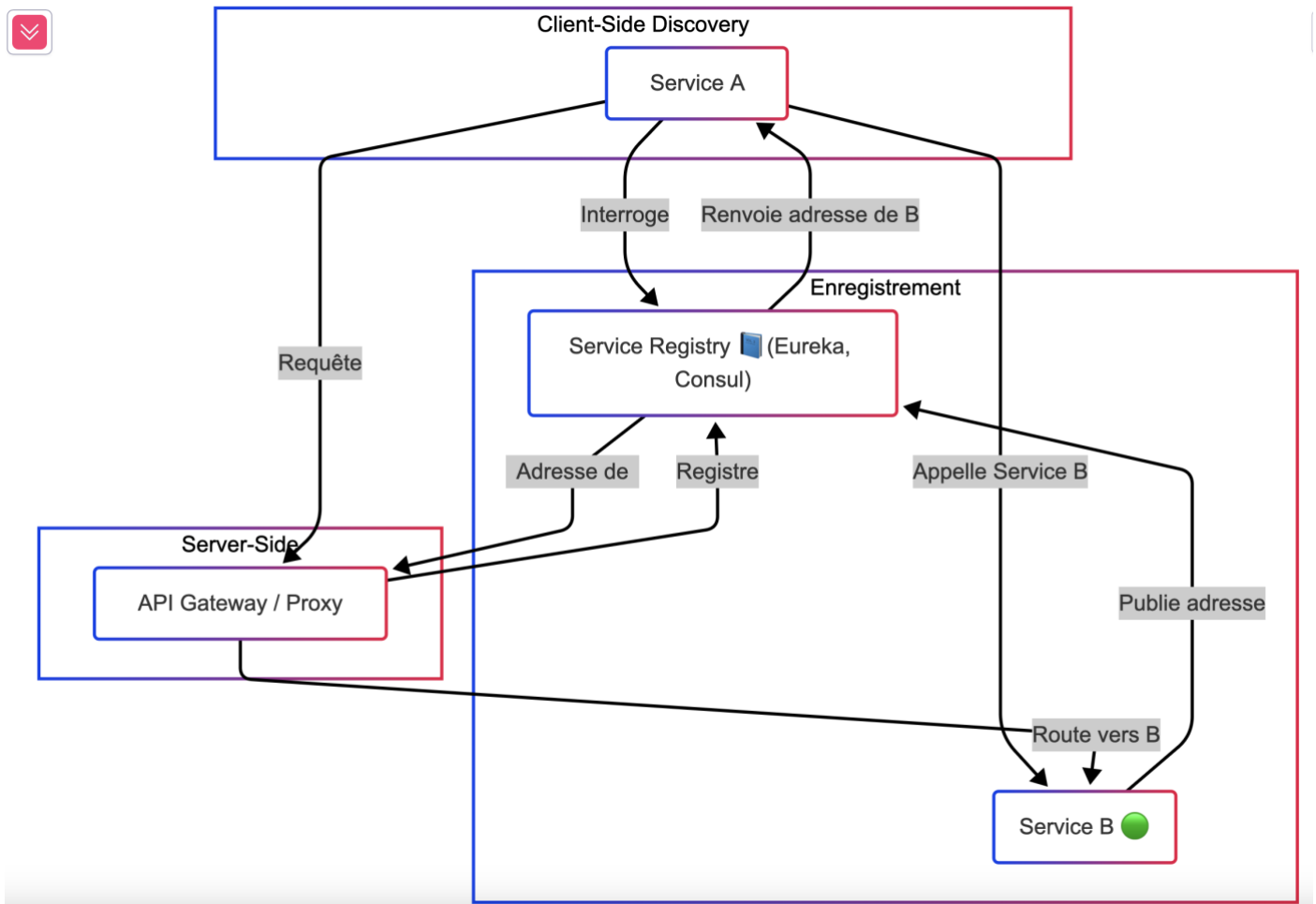
Avantage : plus de contrôle côté client.

Inconvénient : complexité du client, code à maintenir.

- **Server-Side Discovery** :
 - Le client envoie sa requête à un proxy ou une passerelle (ex. : API Gateway).
 - C'est cette passerelle qui interroge le registre et redirige dynamiquement la requête vers un service disponible.
 - Exemples : Kubernetes (kube-dns, CoreDNS), Envoy, Istio Pilot.

Avantage : clients plus simples, logique centralisée.

Inconvénient : dépendance à l'infrastructure (service mesh, gateway).



Cas d'usage avec Kubernetes

Dans Kubernetes, les pods (instances de services) sont souvent éphémères. Kubernetes utilise :

- **CoreDNS** pour résoudre les noms des services vers leurs adresses IP dynamiques.
- **Services virtuels (ClusterIP, NodePort, LoadBalancer)** pour exposer les services internes ou externes.
- Les labels et sélecteurs permettent de router les requêtes vers les bons pods.

Exemples d'outils de Service Discovery

- **Eureka** (Netflix)
- **Consul** (HashiCorp)
- **Zookeeper** (Apache)
- **Kubernetes DNS** (CoreDNS)
- **Nacos** (Alibaba)

2.2 API Gateway

Rôle central

Dans une architecture microservices, l'API Gateway joue un rôle fondamental : il s'agit du **point d'entrée unique** pour toutes les requêtes venant de l'extérieur (applications clientes, navigateurs, systèmes tiers). Plutôt que d'exposer chaque microservice individuellement, l'API Gateway centralise les interactions et agit comme un **proxy inverse intelligent**, capable de router, filtrer, transformer et sécuriser les communications.

Cette couche intermédiaire facilite la gestion, améliore la sécurité et permet une meilleure évolutivité du système.

Fonctions clés

- **Routage dynamique :**
 - Acheminement des requêtes vers le bon microservice selon l'URL, la méthode HTTP, le contenu ou d'autres règles métier.
 - Support de versioning d'API (ex : /v1/produits → Service A, /v2/produits → Service B).
- **Sécurité :**
 - Vérification d'identité (authentification) via OAuth2, JWT, etc.
 - Gestion des rôles et des permissions (autorisation).
 - Protection contre les attaques : throttling, rate limiting, anti-DDoS.
- **Agrégation de données :**
 - Réunification de réponses provenant de plusieurs microservices (pattern Backends for Frontends).
 - Réduction du nombre d'appels réseau côté client (optimisation mobile ou frontend).
- **Transformation de protocole :**
 - Conversion REST ↔ gRPC, ou JSON ↔ XML.
 - Adaptation du format de message selon le client ou le backend.
- **Monitoring et logging centralisé :**
 - Journalisation des appels, mesure des temps de réponse, suivi des erreurs.

Cas d'usage concret

Imaginons une application mobile de livraison. Au lieu de faire 3 appels séparés pour récupérer le profil utilisateur, la liste de commandes et les promotions en cours, l'API Gateway expose un **endpoint agrégé** /home qui rassemble toutes les données nécessaires en un seul appel.

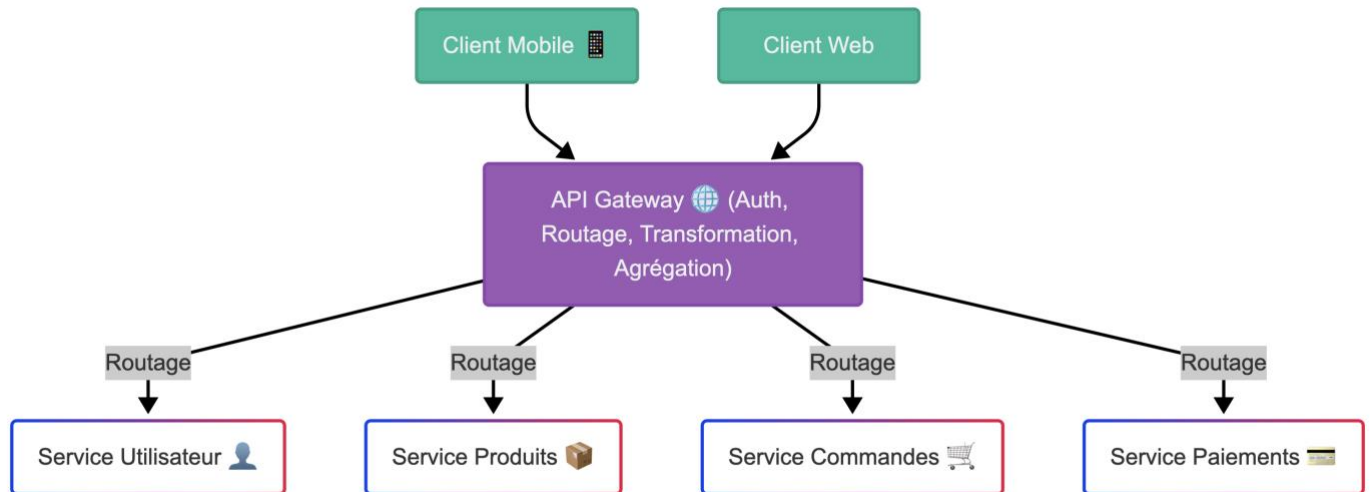
Cela permet de :

- Réduire le temps de réponse visible par l'utilisateur final.
- Alléger la logique dans le client mobile.
- Optimiser la bande passante et les ressources serveur.

Outils populaires

Voici quelques solutions open source ou commerciales largement utilisées en production :

- **NGINX** : léger, rapide, configurable en reverse proxy.
- **Kong** : extensible via des plugins, support natif d'OAuth2, rate limiting, etc.
- **Zuul** (Netflix) : API Gateway Java basé sur la JVM.
- **Ambassador** : API Gateway Kubernetes-native basé sur Envoy.
- **Apigee** (Google) : solution complète avec console de gestion, analytics, sécurité.
- **Traefik** : très populaire dans les environnements Docker/Kubernetes.



2.3 Configuration Management

Besoins

Dans une architecture microservices, chaque service fonctionne de manière autonome et nécessite des **paramètres de configuration spécifiques** pour fonctionner correctement dans différents environnements (développement, test, production, etc.). Ces paramètres incluent :

- **Variables d'environnement** : pour définir le comportement des services selon le contexte d'exécution.
- **Ports d'écoute** pour les communications inter-services.
- **URL de services externes** ou d'autres microservices (ex : base de données, service d'authentification).
- **Clés API et tokens d'authentification**.
- **Secrets sensibles** : mots de passe, certificats, credentials.

Enjeux

Gérer efficacement la configuration dans un environnement distribué présente plusieurs défis :

- Comment appliquer une configuration cohérente à un ensemble de services ?
- Comment éviter les duplications et garantir la sécurité ?
- Comment permettre des mises à jour de configuration sans interrompre les services ?

Bonnes pratiques

- **Centralisation de la configuration** : Utiliser un serveur centralisé de configuration permet de stocker et de distribuer les paramètres à tous les services de manière uniforme.
 - Exemples : Spring Cloud Config, HashiCorp Vault, Consul.
- **Sécurisation des secrets** : Les informations sensibles (mots de passe, tokens, clés privées) ne doivent jamais être codées en dur dans le code ou dans des fichiers de configuration.
 - Utiliser des coffres-forts numériques comme Vault pour gérer les accès et les rotations automatiques.
- **Hot reload et mises à jour dynamiques** : Dans un système hautement disponible, il est souhaitable de mettre à jour la configuration **sans redémarrage** des services. Certains systèmes comme Spring Cloud Bus permettent de notifier automatiquement les services des changements.
- **Configuration par environnement** :
 - Isoler les paramètres spécifiques à chaque environnement (dev, staging, prod) dans des fichiers ou branches dédiés.
 - Utiliser des conventions de nommage ou des profils de configuration pour basculer facilement entre les environnements.

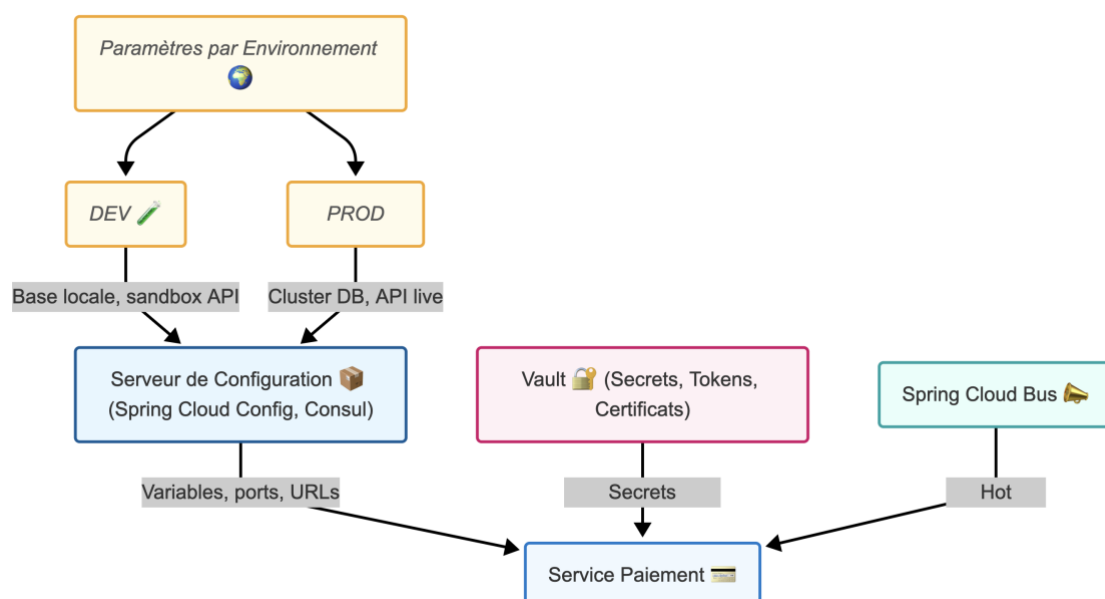
Exemple concret

Dans un projet de microservices e-commerce, le service "Paiement" doit accéder à une base de données PostgreSQL, à une API de traitement de carte bancaire, et utiliser un token OAuth2 pour la sécurité.

- Environnement DEV : base locale, sandbox API, token générique.
- Environnement PROD : base en cluster, API live, token chiffré dans Vault.

L'utilisation d'un serveur de configuration externe permet de :

- Modifier les endpoints API sans toucher au code.
- Appliquer les changements à tous les services automatiquement via hot reload.
- Gérer les droits d'accès aux secrets de manière centralisée.



III. Communication entre Microservices

3.1 Communication synchrone (REST, gRPC)

La communication **synchrone** implique qu'un **service A** effectue une requête à **un service B** et **attend une réponse immédiate**. Cette approche est couramment utilisée dans des contextes où le traitement doit être immédiat ou en temps réel, comme les appels d'API REST ou gRPC. Bien que simple à mettre en œuvre, ce modèle impose que les deux services soient disponibles et réactifs au même moment, ce qui crée un couplage temporel.

REST (Representational State Transfer)

- Basé sur le protocole HTTP/1.1, REST est une norme architecturale simple et largement adoptée.
- Utilise des méthodes HTTP standards : GET, POST, PUT, DELETE.
- Les données sont échangées au format JSON ou XML.
- Avantage : simplicité, compatibilité avec presque tous les outils modernes.
- Idéal pour l'interopérabilité entre services internes et externes.

Exemple : Un service "Commande" effectue une requête GET vers le service "Stock" pour vérifier la disponibilité des produits. La réponse JSON contient les quantités en stock.

gRPC (Google Remote Procedure Call)

- Développé par Google, basé sur HTTP/2, gRPC utilise les Protocol Buffers (protobuf) pour la sérialisation des messages.
- Très performant : connexion persistante, compression des données, multiplexage des appels.
- Supporte les appels unaires, le streaming côté client, côté serveur ou bidirectionnel.
- Fortement typé : chaque message et chaque service sont définis dans des fichiers .proto.
- Idéal pour la communication interne entre services avec des exigences de performance élevées.

Exemple : Le service "Paiement" appelle en gRPC le service "Banque" pour initier une transaction. Le message contient les informations bancaires, le montant, et la réponse est structurée sous forme de protocole binaire.

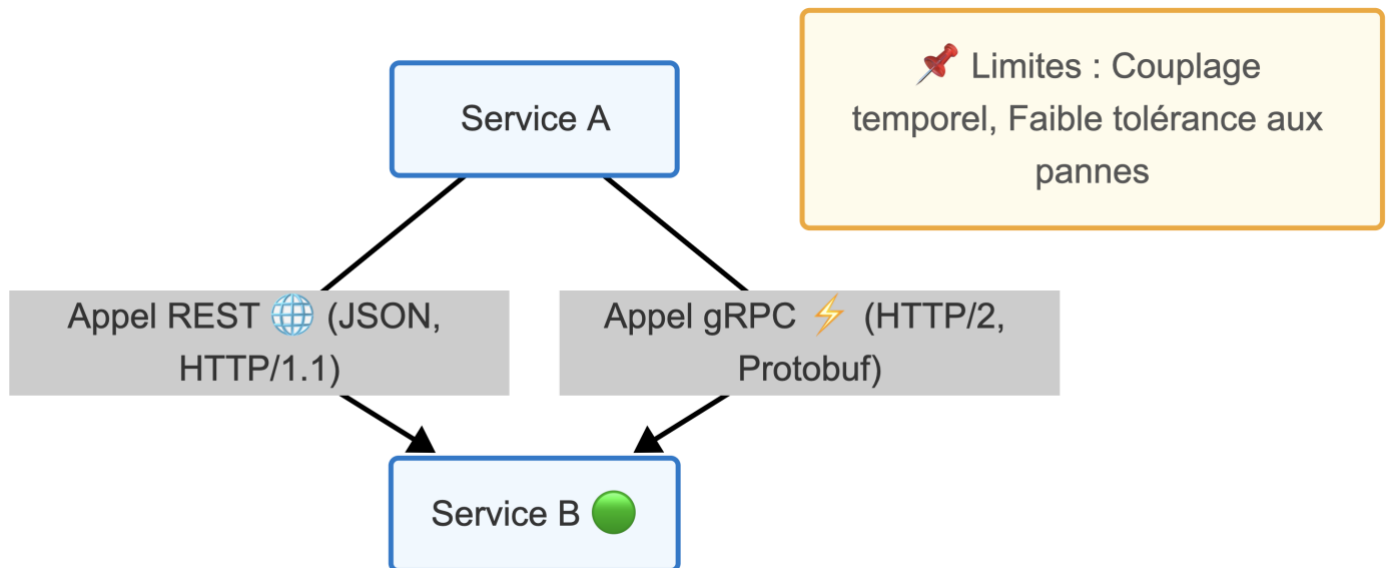
Comparaison REST vs gRPC

Caractéristique	REST	gRPC
Protocole	HTTP/1.1	HTTP/2
Format de données	JSON (texte)	Protobuf (binaire)
Performance	Moyenne	Élevée
Simplicité	Très simple	Plus complexe à mettre en œuvre
Streaming	Non supporté nativement	Support complet
Idéal pour	API publiques, tiers externes	Communication interne rapide

Limites communes

- **Couplage temporel :** le service appelant doit attendre que le service cible réponde. Cela réduit la résilience globale du système.

- **Propagation des pannes** : si un service ne répond pas, cela peut bloquer une chaîne complète d'appels.
- **Gestion des erreurs** : nécessite la mise en place de timeouts, retries, et mécanismes de fallback (circuit breaker).



3.2 Communication asynchrone (messaging)

Principe

Contrairement à la communication synchrone, la communication **asynchrone** ne nécessite pas que les services soient disponibles au même moment. Les microservices communiquent en s'échangeant des **événements** ou des **messages** via un composant intermédiaire appelé **message broker**. Le producteur envoie un message à un canal (file, topic), et un ou plusieurs consommateurs le récupèrent et y réagissent.

Ce modèle est au cœur de l'**architecture pilotée par les événements** (event-driven), dans laquelle les services réagissent à des changements d'état signalés par des événements.

Outils de messagerie

- **RabbitMQ** :
 - Basé sur le protocole AMQP (Advanced Message Queuing Protocol).
 - Fonctionnement via files d'attente.
 - Très fiable pour le traitement séquentiel des messages.
 - Garantit l'ordre de traitement (FIFO).
- **Apache Kafka** :
 - Basé sur la notion de **topics partitionnés**.
 - Hautement performant, scalable horizontalement.
 - Idéal pour les flux de données à fort volume (logs, événements utilisateurs).
 - Permet la **relecture des événements**.

Schéma général

1. Le producteur (Publisher) envoie un message (événement) vers un **broker**.
2. Le broker stocke et distribue les messages.
3. Un ou plusieurs consommateurs (Subscribers) les traitent à leur rythme.

Avantages

- **Découplage temporel** : le producteur et le consommateur ne sont pas obligés d'être connectés simultanément.
- **Haute résilience** : les messages sont conservés même si un service consommateur est temporairement indisponible.
- **Scalabilité native** : plusieurs consommateurs peuvent s'abonner au même flux (fan-out).
- **Extensibilité** : facile d'ajouter un nouveau service qui écoute un événement existant.

Cas d'usage courant

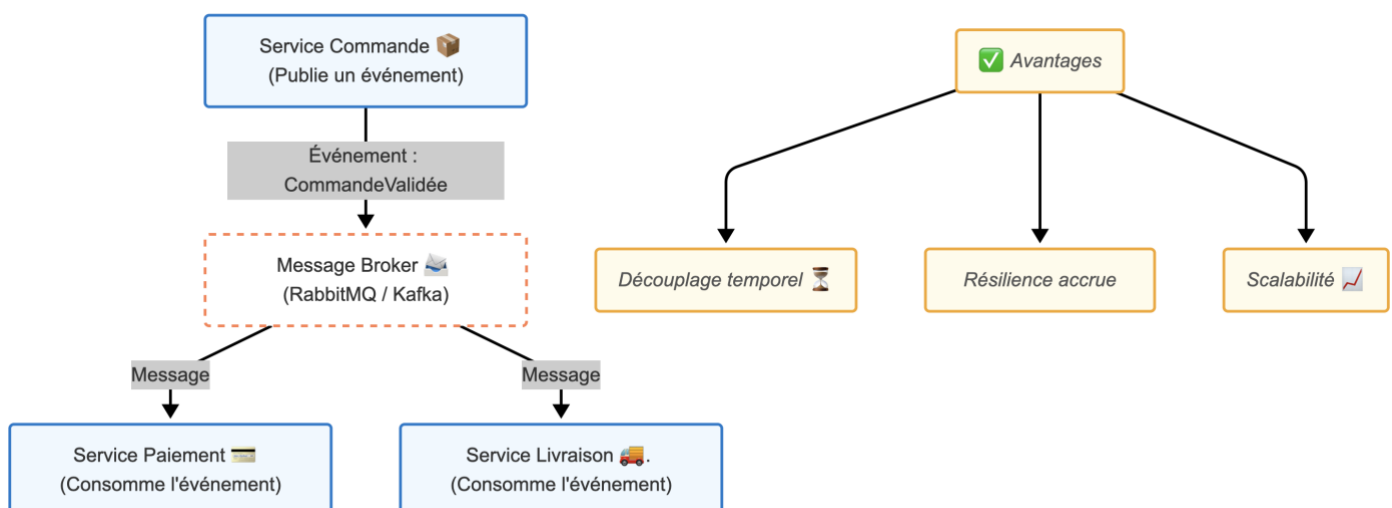
Prenons une application e-commerce.

- Le **service Commande** publie un événement *CommandeValidée*.
- Le **service Paiement** consomme cet événement pour initier la transaction bancaire.
- Le **service Livraison** consomme le même événement pour préparer l'expédition.
- Le **service Notification** envoie un e-mail de confirmation.

Chacun de ces services agit indépendamment, ce qui permet une forte tolérance aux pannes et une évolutivité accrue.

Inconvénients / défis

- **Complexité de débogage** : plus difficile à tracer que les appels directs.
- **Garantie d'ordre** : parfois difficile à maintenir, notamment avec plusieurs partitions.
- **Cohérence des données** : nécessite de gérer l'état de manière distribuée.
- **Gestion des duplications** : nécessite des mécanismes d'idempotence.



3.3 Patterns et bonnes pratiques

Dans un système basé sur les microservices, il ne suffit pas de choisir une méthode de communication (REST, messaging, etc.) : il est essentiel d'appliquer des **patterns éprouvés** qui garantissent la fiabilité, la résilience et la lisibilité du système distribué. Voici les principaux patterns et bonnes pratiques à suivre :

Request/Response (REST/gRPC)

Ce pattern classique implique que le client envoie une requête à un service et attend une réponse immédiate. Il est couramment utilisé pour les opérations transactionnelles simples et les échanges synchrones.

- **REST** est simple, universel, basé sur HTTP/1.1 et JSON.
- **gRPC** est plus rapide et typé, idéal pour les communications internes performantes.

Ce modèle est utile lorsque la cohérence immédiate des données est nécessaire, mais il introduit un **couplage temporel fort** et peut poser des problèmes en cas de panne.

Event-Driven (pub/sub)

Dans une architecture pilotée par les événements, les services ne s'appellent pas directement. Au lieu de cela, un producteur publie un événement sur un bus (ex : Kafka), et un ou plusieurs consommateurs y réagissent.

- Permet un **découplage fort**, à la fois temporel et logique.
- Favorise la scalabilité et l'extensibilité.
- Nécessite une bonne gestion de l'ordre et de la persistance des événements.

Event-Carried State Transfer

Il s'agit d'une variante avancée de l'event-driven où chaque événement contient **l'état complet du domaine** nécessaire au traitement. Ainsi, les consommateurs ne dépendent pas d'un appel synchrone complémentaire pour obtenir les données.

- Réduit la dépendance entre services.
- Permet une meilleure résilience et une plus grande tolérance aux pannes.
- Utilisé dans des architectures orientées CQRS ou dans des workflows asynchrones complexes.

Idempotence

Un appel idempotent garantit que **répéter plusieurs fois la même requête produit le même effet** qu'un seul appel. C'est essentiel dans les systèmes distribués où les doublons peuvent apparaître (ex : après un timeout).

- Appliqué notamment sur les endpoints PUT, DELETE, ou dans la consommation d'événements.
- Implémentation via identifiants uniques, verrous logiques ou contrôle d'état métier.

Timeouts et retries

Dans un système distribué, certaines requêtes échouent ou expirent. Il est donc crucial d'appliquer :

- **Timeouts** : ne pas attendre indéfiniment une réponse.
- **Retries** : retenter automatiquement une opération avec stratégie d'attente (exponential backoff).

⚠ Attention : sans contrôle, les retries peuvent aggraver une situation de surcharge. Il faut combiner avec un **circuit breaker**.

Circuit Breaker

Ce pattern protège les services en arrêtant temporairement les appels vers un service distant en échec. Il agit comme un disjoncteur :

- **État fermé** : les appels passent normalement.
- **État ouvert** : les appels échouent immédiatement.
- **État semi-ouvert** : tests progressifs pour rétablir la connexion.

Il permet d'éviter l'effet domino et de préserver les ressources disponibles.

Outils : Hystrix (Netflix), Resilience4j, Istio + Envoy.

IV. Concepts avancés

4.1 Architecture pilotée par les événements : CQRS, Saga

Les architectures pilotées par les événements sont devenues essentielles dans les systèmes distribués modernes. Elles permettent une meilleure réactivité, une résilience accrue et une décomposition claire des responsabilités. Deux patterns fondamentaux dans ce paradigme sont **CQRS** et **Saga**.

CQRS (Command Query Responsibility Segregation)

Le pattern CQRS vise à **séparer les responsabilités entre les lectures et les écritures** dans un système. Cela permet d'optimiser indépendamment les performances des opérations de lecture (query) et d'écriture (command), ce qui est particulièrement utile dans des systèmes à haute charge ou à forte scalabilité.

- **Command** : représente une action ou une intention de changement de l'état (ex : "Créer une commande").
- **Query** : représente une lecture de données sans effet de bord (ex : "Lister les commandes du jour").

Avantages :

- Meilleure scalabilité : les lectures peuvent être réparties différemment des écritures.
- Meilleure performance : on peut optimiser la base de données de lecture pour la rapidité (ex : cache, vues matérialisées).
- Modèles de données spécialisés : chaque côté peut avoir son propre modèle.

Exemple :

Dans une application e-commerce, le service de **commande** reçoit une requête POST pour créer une commande (Command). En parallèle, une base optimisée (NoSQL ou Elasticsearch) expose une API GET pour rechercher les commandes passées (Query).

Saga Pattern

Dans un système distribué, une transaction globale (ex : réserver un vol, payer, envoyer un e-mail) implique plusieurs microservices. Le **Saga pattern** est un moyen de gérer ces **transactions longues et distribuées** sans avoir recours à un verrou global ou à un commit distribué (2PC).

Un Saga est composé d'une **séquence d'étapes locales**, chacune étant une transaction dans un service donné. Si l'une échoue, des **opérations compensatoires** sont déclenchées.

Deux modèles principaux :

- **Orchestration** : un orchestrateur central (ex : un moteur de workflow) contrôle la séquence des appels et gère les erreurs.

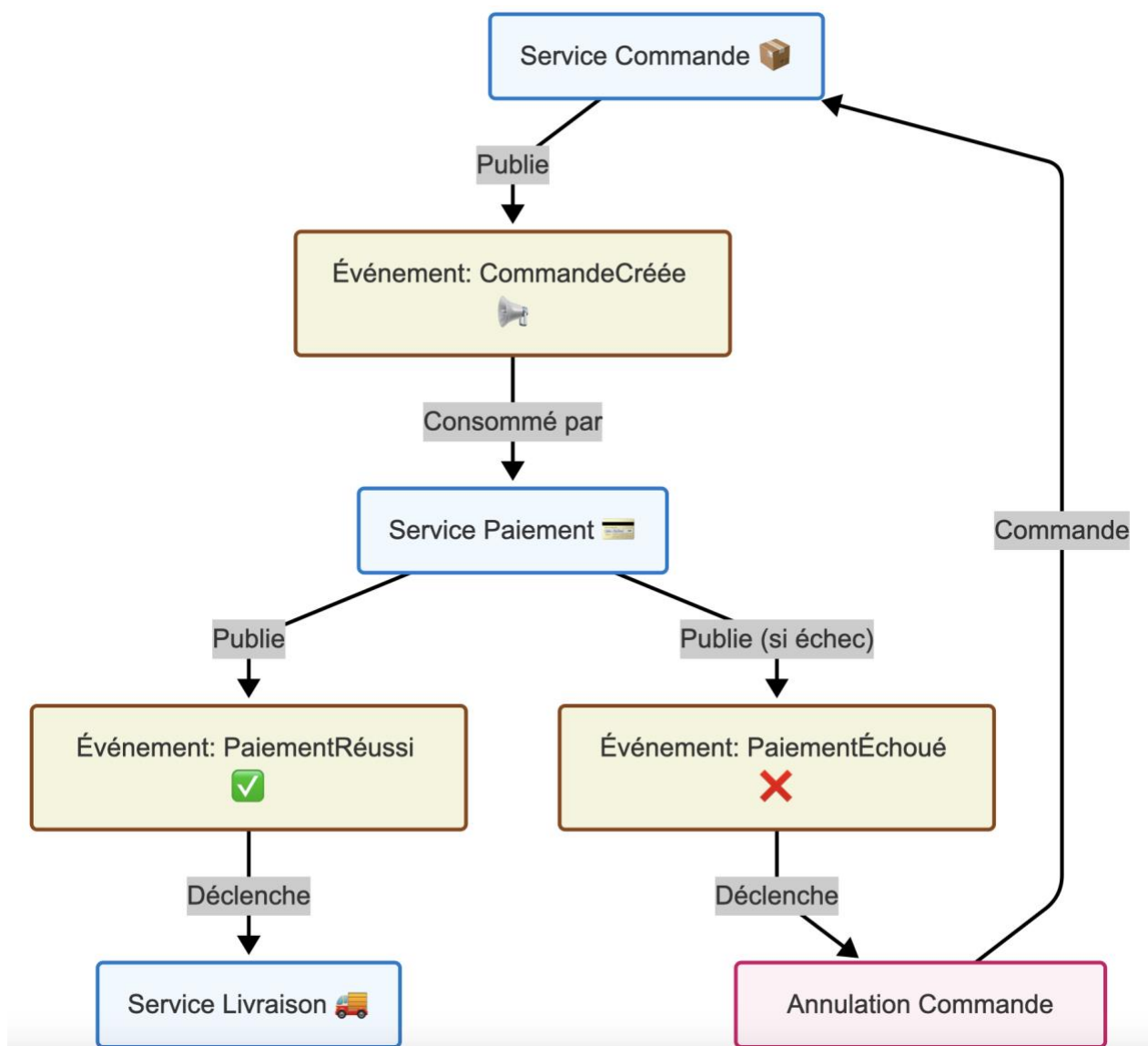
- Avantage : visibilité centralisée, logique explicite.
- Inconvénient : couplage fort avec l'orchestrateur.
- **Chorégraphie** : chaque service écoute des événements et répond par des actions ou la publication de nouveaux événements.
 - Avantage : forte décentralisation, meilleure scalabilité.
 - Inconvénient : logique plus difficile à suivre et tester.

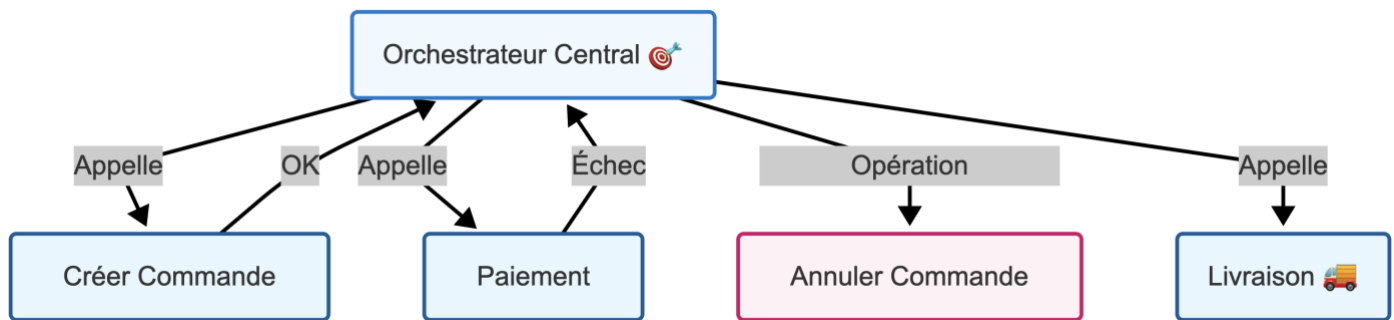
Exemple :

- **Service Commande** : crée une commande → publie CommandeCréée
- **Service Paiement** : reçoit l'événement → tente le paiement → publie PaiementRéussi
- **Service Livraison** : déclenche l'expédition en réponse à PaiementRéussi

Si le paiement échoue, un événement PaiementÉchoué est émis et peut entraîner une annulation côté Commande.

Exemple Chorégraphie:



Exemple Orchestration:**4.2 Résilience et Scalabilité**

Les systèmes distribués sont plus exposés aux défaillances que les applications monolithiques, notamment à cause de la multiplicité des points de défaillance (réseau, service distant, base de données, etc.). Deux mécanismes essentiels pour rendre une architecture microservices plus robuste et scalable sont : le **circuit breaker** et le **service mesh**.

Circuit Breaker

Le **circuit breaker** (ou disjoncteur applicatif) est un pattern de résilience qui vise à **éviter la propagation des pannes** dans un système distribué. Lorsqu'un service distant est instable ou indisponible, le circuit breaker bloque temporairement les appels vers ce service afin de protéger le reste du système et d'éviter l'engorgement.

Fonctionnement :

- **État fermé** : tout fonctionne normalement, les requêtes passent.
- **État ouvert** : après un certain nombre d'échecs consécutifs, les appels sont automatiquement interrompus pendant un délai défini.
- **État semi-ouvert** : après une période de repos, quelques requêtes tests sont autorisées pour vérifier si le service est rétabli.

Avantages :

- Réduit la latence perçue en évitant les appels voués à échouer.
- Protège les services consommateurs de surcharge inutile.
- Permet une reprise contrôlée du trafic une fois le service rétabli.

Outils :

- **Hystrix** (Netflix)
- **Resilience4j** (Java)
- **Istio / Envoy** (intégration côté infrastructure)

Service Mesh

Un **service mesh** est une couche d'infrastructure dédiée à la gestion automatique de la **communication inter-services**. Il se compose de **sidecars** (ex. : Envoy) injectés dans chaque pod, et d'un **plan de contrôle** (ex. : Istio Pilot).

Fonctionnalités principales :

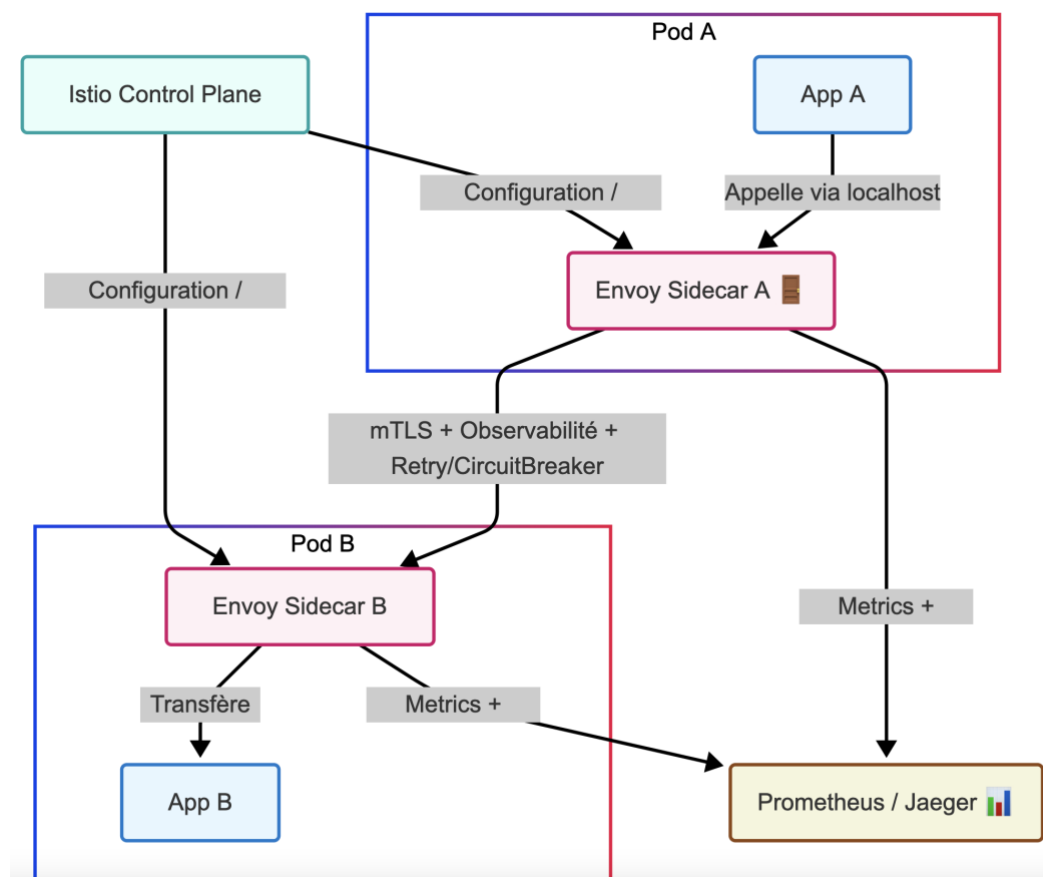
- **Sécurité par défaut** : chiffrement TLS/mTLS automatique entre services.
- **Retries, timeouts, circuit breakers configurables** sans changer le code.
- **Observabilité native** : collecte de métriques (Prometheus), traçage (Jaeger).
- **Traffic shaping** : routage conditionnel, canary releases, mirroring.
- **Contrôle d'accès** (RBAC) entre services.

Exemples :

- **Istio** : service mesh complet et extensible, basé sur Envoy.
- **Linkerd** : léger, conçu pour la simplicité et la rapidité.
- **Consul Connect** : ajout de mTLS et politiques à Consul.

Avantages :

- Centralisation de la configuration réseau.
- Découplage entre logique applicative et logique de communication.
- Sécurité renforcée entre les microservices sans effort de développement.



4.3 Monitoring et Tracing

Objectif : Observabilité

Dans une architecture distribuée à base de microservices, l'observabilité devient cruciale pour garantir la **visibilité**, la **compréhension** et la **résolution rapide des incidents**. Elle repose sur trois piliers principaux : **les métriques**, **les logs** et **les traces distribuées**.

Outils clés

- **Prometheus :**
 - Système de monitoring open source développé par la CNCF.
 - Collecte de métriques sous forme de séries temporelles : latence, nombre de requêtes, taux d'erreur, usage mémoire/CPU.
 - Utilise un modèle de requêtage puissant : PromQL.
 - Intégration native avec Kubernetes, Istio, Envoy, etc.
- **Grafana :**
 - Plateforme de visualisation de données.
 - Utilisée pour construire des **tableaux de bord interactifs** à partir des métriques de Prometheus.
 - Permet de définir des alertes, des seuils critiques, des annotations.
- **Jaeger :**
 - Outil open source de traçabilité des requêtes distribuées.
 - Permet de suivre un appel utilisateur **à travers plusieurs services**.
 - Affiche les temps de traitement, les dépendances entre services, et les goulots d'étranglement.
 - Essentiel pour le **debugging**, l'analyse de performance et le profiling.

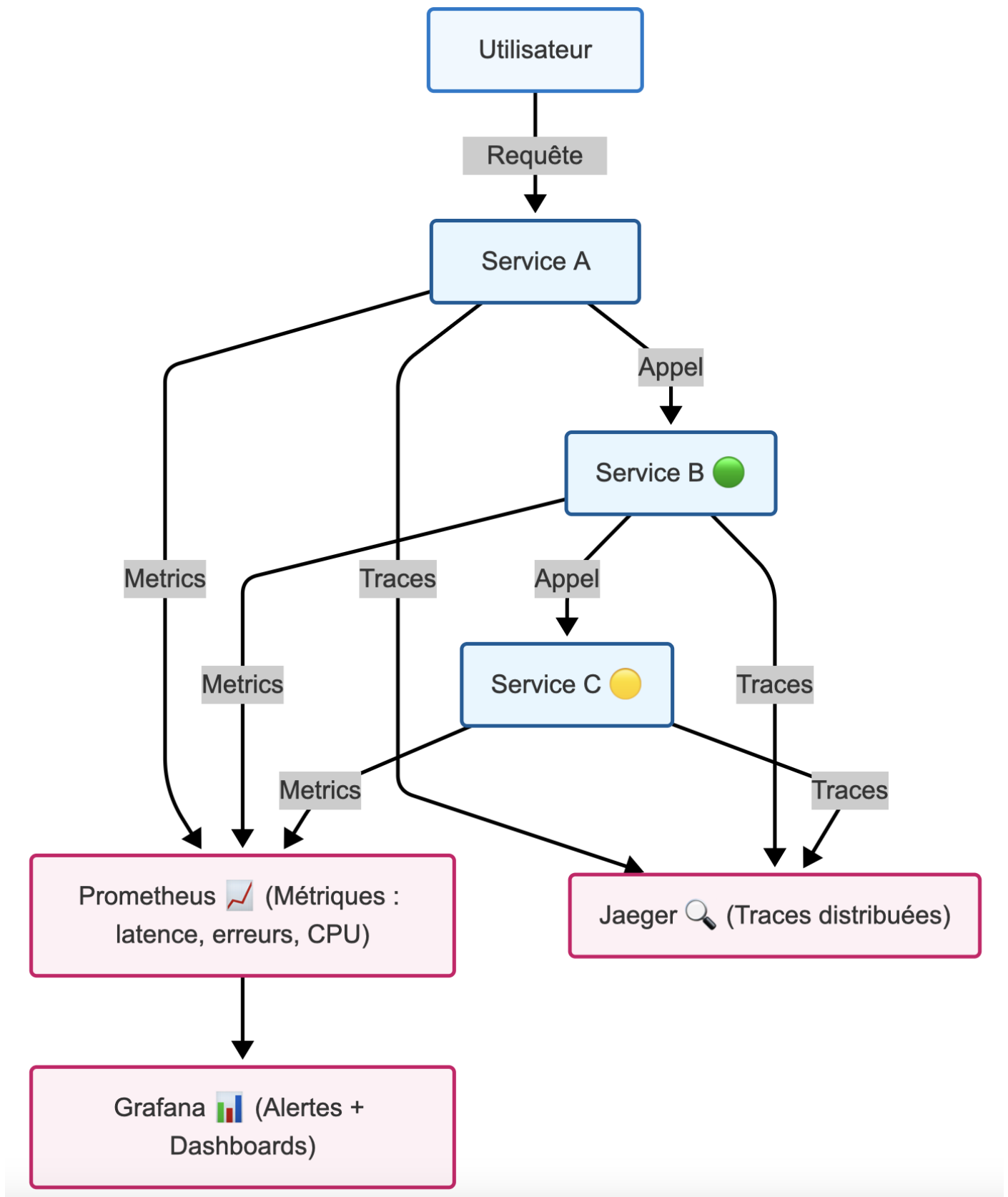
Exemple concret

Dans un système de paiement :

- Prometheus collecte les métriques de latence du service Paiement.
- Grafana alerte si la latence dépasse 1s pour 5% des requêtes.
- Jaeger permet de tracer une requête POST /payer et d'identifier que le ralentissement provient du service de vérification bancaire.

Bénéfices de l'observabilité

- **Détection proactive des anomalies** (temps de réponse, erreurs, saturation).
- **Analyse post-mortem facilitée** grâce aux traces.
- **Compréhension du comportement des services en production.**
- **Amélioration continue de la performance.**



V. Pipeline historique des microservices

5.1 Ancêtres : RPC, CORBA

RPC (Remote Procedure Call)

Le RPC permettait à un programme d'appeler une fonction située sur une autre machine comme s'il s'agissait d'un appel local. Il offrait une abstraction puissante mais masquait les complexités du réseau, ce qui pouvait conduire à des erreurs difficiles à diagnostiquer.

Avantages :

- Interface simple
- Gain de temps pour les développeurs

Limites :

- Très sensible aux pannes réseau
- Couplage fort entre client et serveur
- Faible tolérance aux erreurs
- Difficulté d'évolution dans des architectures complexes

CORBA (Common Object Request Broker Architecture)

CORBA est un standard défini par l'OMG (Object Management Group) pour permettre la communication entre objets distribués dans des langages et des systèmes différents.

Avantages :

- Forte abstraction grâce à l'utilisation d'un ORB (Object Request Broker)
- Interopérabilité multi-langages avec IDL (Interface Definition Language)

Limites :

- Mise en œuvre lourde et complexe
- Manque de flexibilité
- Difficulté de débogage
- Faible adoption en dehors des grandes entreprises industrielles

5.2 De SOA à Microservices

SOA (Service-Oriented Architecture)

SOA visait à modulariser les systèmes en services réutilisables communiquant via un **Enterprise Service Bus (ESB)**. Chaque service encapsule une fonction métier (facturation, livraison, etc.).

Progrès apportés par SOA :

- Réutilisabilité des services

- Standardisation via SOAP, WSDL
- Décomposition logique par domaines fonctionnels

Limites de SOA :

- **Dépendance au middleware ESB**, devenant un goulot d'étranglement
- Risque de point de défaillance unique
- Surcoût en performance et en complexité de configuration
- Faible adaptabilité aux évolutions rapides

Émergence des Microservices

Les microservices sont apparus comme une réponse moderne aux limites de SOA. Ils conservent l'idée de découpage fonctionnel mais éliminent la centralisation excessive.

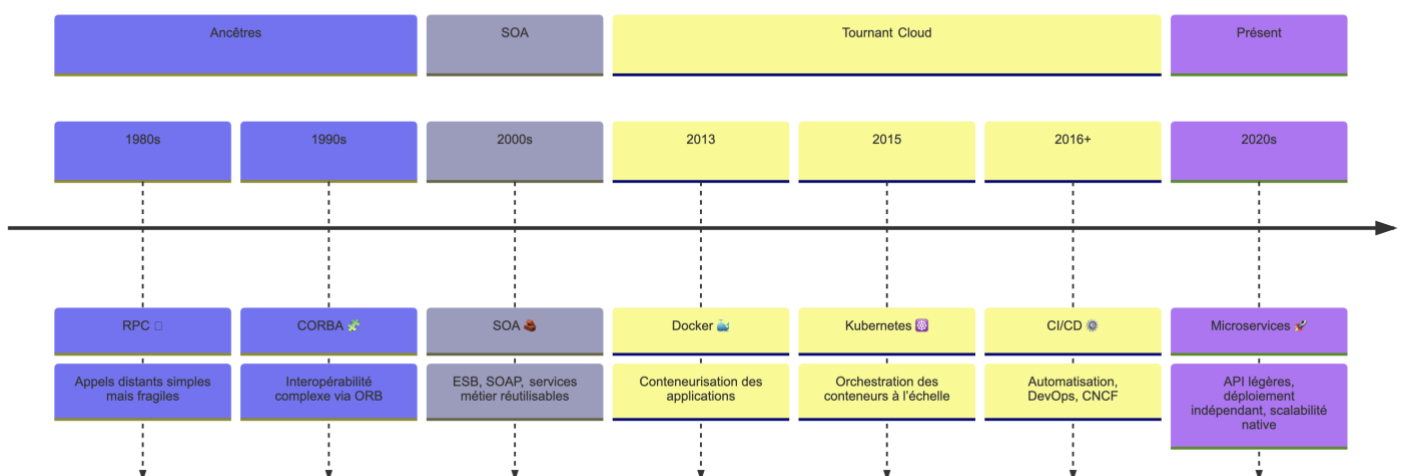
Principes clés :

- Services autonomes et faiblement couplés
- Communication via API légères (REST, gRPC, messaging)
- Déploiement indépendant dans des conteneurs
- Adaptation au cloud, DevOps, CI/CD

Tournants technologiques majeurs

- **2013 – Docker** : démocratise la conteneurisation des applications, facilitant l'isolation et le déploiement rapide.
- **2015 – Kubernetes** : orchestration des conteneurs à grande échelle, automatisation du déploiement, scaling, et gestion des ressources.
- **2016+ – CNCF, DevOps, CI/CD** : formalisation de l'écosystème cloud-native, adoption massive de pipelines d'intégration et de déploiement continus pour livrer plus vite et plus souvent.

Évolution des architectures distribuées



Chapitre 4

Conteneurisation avancée et Orchestration

I. Introduction aux Conteneurs

1. Historique détaillé et évolution des technologies de virtualisation

La virtualisation est une technologie permettant d'exécuter plusieurs systèmes d'exploitation sur un seul serveur physique. Apparu dès les années 1960 avec les mainframes IBM, ce concept évolua significativement avec l'émergence des hyperviseurs tels que VMware ESXi, Xen et Hyper-V à partir des années 1990. Ces hyperviseurs facilitent la gestion des ressources et l'isolation des environnements, favorisant ainsi l'efficacité et la flexibilité opérationnelles.

Vers les années 2000, la technologie de conteneurisation prend forme avec Linux Containers (LXC) qui apportent une virtualisation plus légère grâce au partage du noyau de l'OS hôte. En 2013, Docker popularise l'utilisation des conteneurs grâce à sa simplicité d'utilisation, accélérant considérablement les pratiques DevOps et favorisant des déploiements plus rapides et automatisés.

2. Comparaison approfondie entre Conteneurs et Machines virtuelles

Les différences majeures entre les conteneurs et les machines virtuelles se trouvent dans leur manière d'isoler les applications, leur consommation de ressources, leur rapidité de démarrage, leur portabilité et leur capacité d'évolution :

Critères	Machines Virtuelles	Conteneurs
Isolation	Isolation forte via OS complet dédié	Isolation par namespaces et groupes cgroups
Ressources	Élevée, car chaque VM possède un OS complet	Faible, uniquement applications et bibliothèques partagées
Démarrage	Lent (quelques minutes)	Très rapide (quelques secondes ou moins)
Portabilité	Limitée (dépendante de l'hyperviseur)	Très élevée (grâce à Docker Hub et OCI standards)
Mise à l'échelle	Complexe et onéreuse	Facile, rapide, flexible et économique

II. Concepts fondamentaux Docker

1. Docker : Définition détaillée

Docker est une plateforme open-source destinée à simplifier le processus de création, d'expédition et d'exécution des applications via des conteneurs. Un conteneur encapsule l'application avec toutes ses dépendances (bibliothèques, outils, fichiers binaires, etc.), assurant ainsi une cohérence parfaite entre les environnements de développement, test et production.

2. Docker : Terminologie détaillée

- **Image Docker** : Modèle immuable servant de base à la création de conteneurs. Comparable à une classe en programmation orientée objet.
- **Conteneur Docker** : Instance exécutable d'une image Docker. Semblable à une instance d'objet issue d'une classe.
- **Dockerfile** : Fichier texte décrivant précisément les étapes pour construire une image Docker (instructions telles que FROM, RUN, COPY, CMD, etc.).
- **Docker Hub** : Registre officiel en ligne regroupant un grand nombre d'images Docker officielles et communautaires, facilitant le partage et la collaboration.

3. Commandes Docker essentielles expliquées en profondeur

- **Lister les images Docker disponibles localement :**

```
docker images
```

- **Créer et démarrer un conteneur détaché (background) :**

```
docker run -d -p 80:80 nginx
```

- **Afficher les conteneurs actifs :**

```
docker ps
```

- **Lister tous les conteneurs (actifs et arrêtés) :**

```
docker ps -a
```

- **Arrêter un conteneur spécifique :**

```
docker stop <container_id>
```

- **Supprimer un conteneur :**

```
docker rm <container_id>
```

- **Créer une image Docker à partir d'un Dockerfile :**

```
docker build -t nom_image .
```

4. Exemple pratique complet avec explications détaillées

Pour exécuter un serveur web Nginx simplement :

1. Démarrer le conteneur :

```
docker run -d -p 8080:80 --name serveur_web nginx
```

- `-d` lance le conteneur en arrière-plan.

- -p 8080:80 redirige le port 80 du conteneur vers le port 8080 de l'hôte.
- --name serveur_web nomme explicitement le conteneur.

2. Vérifier l'accès au serveur via navigateur :

- URL : `http://localhost:8080`

3. Arrêter puis supprimer le conteneur :

```
docker stop serveur_web
docker rm serveur_web
```

Cette démonstration pratique illustre clairement le workflow typique avec Docker.

III. Techniques avancées Docker

1. Gestion avancée des réseaux Docker

Docker permet aux conteneurs de communiquer entre eux ou avec le monde extérieur via différents types de réseaux personnalisables.

- **Bridge (pont)** : Réseau par défaut. Chaque conteneur reçoit une adresse IP privée et peut communiquer avec les autres sur le même réseau.

```
docker network create --driver bridge mon_reseau
```

- **Host** : Le conteneur utilise directement l'interface réseau de l'hôte. Utile pour les performances ou cas spécifiques (ex : services réseau).

```
docker run --network host nginx
```

- **Overlay** : Utilisé pour interconnecter des conteneurs situés sur des hôtes différents (souvent dans Docker Swarm).
- **Macvlan** : Attribue une adresse MAC unique au conteneur. Utilisé dans des environnements où une intégration directe au réseau physique est requise.

Exemple de mise en place personnalisée :

```
# Créer un réseau bridge personnalisé
docker network create --driver bridge web_net

# Lancer deux conteneurs connectés à ce réseau
docker run -dit --name backend --network web_net alpine

docker run -dit --name frontend --network web_net alpine
```

2. Gestion avancée des volumes Docker

Les volumes sont recommandés pour stocker les données générées et utilisées par les conteneurs. Contrairement aux liaisons de dossiers de l'hôte, ils sont gérés nativement par Docker, ce qui les rend plus portables et sûrs.

Créer et utiliser un volume :

```
docker volume create data_volume
docker run -d -v data_volume:/var/lib/mysql mysql
```

Lister les volumes existants :

```
docker volume ls
```

Inspecter un volume :

```
docker volume inspect data_volume
```

3. Dockerfile avancé et construction Multi-étapes

Un Dockerfile multi-étapes permet d'utiliser plusieurs phases de build dans un même fichier pour produire des images plus légères et sécurisées.

Exemple :

```
# Étape de compilation
FROM node:18-alpine as builder
WORKDIR /app
COPY . .
RUN npm install && npm run build

# Étape de production
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
```

Avantages :

- Réduction de la taille de l'image finale
- Séparation claire entre build et exécution

4. Docker Compose détaillé

Docker Compose permet de définir une application multi-conteneurs via un fichier YAML.

Exemple concret :

```
version: '3.9'
services:
  app:
    build: .
    ports:
      - "3000:3000"
    volumes:
      - ./app
  redis:
    image: redis:alpine
```

Commandes utiles :

- `docker-compose up -d` : lance les services
- `docker-compose down` : arrête et supprime les services
- `docker-compose logs` : affiche les logs

Avantages de Docker Compose :

- Simplifie la configuration et le déploiement
- Très utile en développement et en CI/CD
- Gère dépendances, réseaux, et volumes facilement

IV. Introduction détaillée à Kubernetes

1. Historique de Kubernetes

Kubernetes a été initialement développé par Google, inspiré de son système interne Borg, pour répondre à la gestion massive de conteneurs en production. Il a été publié en open-source en 2014 et est rapidement devenu la norme de facto pour l'orchestration des conteneurs, soutenu par la Cloud Native Computing Foundation (CNCF).

Aujourd'hui, Kubernetes est utilisé par les grandes entreprises pour automatiser le déploiement, la montée en charge (scaling), et la gestion des conteneurs sur des clusters de machines physiques ou virtuelles.

2. Concepts fondamentaux de Kubernetes

- **Pod** : L'unité de base dans Kubernetes. Un pod peut contenir un ou plusieurs conteneurs partageant les ressources réseau et stockage.
- **Node** : Une machine (virtuelle ou physique) qui exécute les pods.
- **Cluster** : Ensemble de nœuds (Nodes) orchestrés par Kubernetes.

- **Service** : Point d'accès stable aux pods, permettant la découverte de service (Service Discovery) et le load balancing.
- **Deployment** : Définit la stratégie de déploiement et de mise à jour des pods. Assure l'autoscaling et le rollback.
- **Namespace** : Utilisé pour segmenter les ressources au sein d'un même cluster.

3. Architecture de Kubernetes

- **Master Node** : Coordonne le cluster. Il exécute des composants comme :
 - kube-apiserver : point d'entrée pour l'API REST.
 - etcd : base de données clé-valeur pour stocker l'état du cluster.
 - kube-scheduler : planifie l'exécution des pods sur les nœuds.
 - kube-controller-manager : contrôle l'état du cluster et applique la configuration désirée.
- **Worker Nodes** : Exécutent les applications via :
 - kubelet : agent qui communique avec le Master.
 - kube-proxy : gère la connectivité réseau.
 - container runtime : moteur de conteneurisation (Docker, containerd, CRI-O, etc.).

4. Installation locale avec Minikube

Minikube permet de créer un cluster Kubernetes local pour l'apprentissage et les tests.

Installation :

```
minikube start
```

Commandes de base :

```
kubectl get pods          # liste les pods
kubectl get services      # liste les services
kubectl apply -f fichier.yaml # applique une configuration
kubectl delete pod nom    # supprime un pod spécifique
```

5. Déploiement avec YAML : exemple

Exemple de fichier de déploiement simple :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: webapp
  template:
    metadata:
      labels:
        app: webapp
    spec:
```

```
containers:
- name: webapp
  image: nginx
  ports:
  - containerPort: 80
```

Déploiement :

```
kubectl apply -f deployment.yaml
```

6. Helm : gestionnaire de packages Kubernetes

Helm permet de déployer des applications Kubernetes à l'aide de packages appelés "charts".

Commandes utiles :

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm install monapp bitnami/nginx
```

Avantages de Helm :

- Simplifie le déploiement d'applications complexes
- Gestion des versions et rollback facile
- Réutilisation des configurations via des templates

V. CI/CD et automatisation avec Kubernetes**1. Historique et importance du CI/CD**

Le CI/CD (Intégration Continue / Déploiement Continu) est un ensemble de pratiques qui visent à automatiser les étapes de build, test et déploiement des applications. L'objectif est d'accélérer la mise en production, améliorer la qualité logicielle, et réduire les erreurs humaines.

- **Intégration continue (CI)** : Automatisation des tests unitaires, builds, et analyse de code à chaque modification de code.
- **Déploiement continu (CD)** : Livraison automatisée des modifications validées vers un environnement de staging ou production.

Avantages du CI/CD :

- Réduction du temps de mise sur le marché (time-to-market)
- Diminution des risques grâce à des tests automatisés
- Livraison incrémentale plus fiable

2. Outils CI/CD populaires compatibles avec Kubernetes

- **Jenkins** : Serveur d'automatisation très personnalisable.
- **GitLab CI/CD** : Intégré dans GitLab avec pipelines YAML.
- **GitHub Actions** : CI/CD natif pour les projets hébergés sur GitHub.

- **ArgoCD** : Pour le déploiement GitOps natif sur Kubernetes.

3. Exemple de pipeline avec GitHub Actions

Ce pipeline construit une image Docker, la pousse sur Docker Hub, puis met à jour le déploiement sur Kubernetes :

.github/workflows/deploy.yaml

```
name: Build and Deploy to Kubernetes
on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v2

      - name: Login to Docker Hub
        run: echo "${{ secrets.DOCKER_PASSWORD }}" | docker login -u
"${{ secrets.DOCKER_USERNAME }}" --password-stdin

      - name: Build and push Docker image
        run: |
          docker build -t "${{ secrets.DOCKER_USERNAME
}}/webapp:latest" .
          docker push "${{ secrets.DOCKER_USERNAME }}/webapp:latest"

      - name: Set up kubectl
        uses: azure/setup-kubectl@v3
        with:
          version: 'latest'

      - name: Deploy to Kubernetes
        run: |
          kubectl set image deployment/webapp webapp=${{
secrets.DOCKER_USERNAME }}/webapp:latest
```

Explication :

- Le pipeline est déclenché à chaque push sur la branche main.
- Il construit une nouvelle image Docker et la pousse sur Docker Hub.
- Il met ensuite à jour le déploiement Kubernetes via kubectl.

4. Déploiement GitOps avec ArgoCD (optionnel)

GitOps est une approche où la configuration de l'infrastructure est stockée dans un dépôt Git et synchronisée automatiquement avec le cluster Kubernetes.

ArgoCD :

- Détecte les changements dans le dépôt Git.
- Applique automatiquement les modifications dans le cluster.
- Fournit une interface graphique pour suivre les statuts des applications.

Commandes de base :

```
argocd app create webapp --repo https://github.com/mon-org/k8s-manifests --path webapp --dest-server https://kubernetes.default.svc --dest-namespace default
```

VI. Surveillance et Auto-Scaling avec Kubernetes

1. Surveillance avec Prometheus et Grafana

La surveillance est cruciale dans un environnement cloud-native. Elle permet de détecter rapidement les anomalies, d'anticiper les incidents et d'optimiser les performances.

- **Prometheus** est un système de surveillance open source qui collecte des métriques via des endpoints HTTP exposés par les services.
 - Stockage de séries temporelles (time-series DB).
 - Langage de requête PromQL.
 - Alertmanager pour la gestion des alertes.
- **Grafana** est un outil de visualisation qui se connecte à Prometheus (et d'autres sources) pour générer des tableaux de bord interactifs.

Exemple de métriques utiles :

- Utilisation CPU et mémoire par Pod
- Nombre de requêtes HTTP traitées
- Temps de réponse moyen

Architecture typique :

Pods → Exporters → Prometheus → Grafana

2. Auto-Scaling avec Kubernetes HPA (Horizontal Pod Autoscaler)

L'auto-scaling permet à Kubernetes d'ajuster dynamiquement le nombre de pods en fonction de la charge système (CPU, mémoire, métriques personnalisées).

- HPA observe les métriques et applique une stratégie de scaling définie.
- Nécessite le Metrics Server installé sur le cluster.

Commandes :

```
# Activer l'autoscaling sur un déploiement
kubectl autoscale deployment webapp --cpu-percent=50 --min=2 --max=10
```



```
# Vérifier les décisions de scaling
kubectl get hpa
```

Configuration YAML exemple :

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: webapp-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: webapp
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
```

Avantages de l'HPA :

- Réduction des coûts en période creuse
- Haute disponibilité en période de charge
- Réaction rapide et automatique aux variations de trafic